MENTOR ROADMAP

.NET Tech Lead

10-day interview roadmap

Backend Web API · C# · Entity Framework Core · SQL Server

Azure App Service · Azure Functions · Microservices · Legacy migration

For a developer with 8 years of .NET experience who is moving into a tech lead role.

Prepared 30 October 2026. Speak about .NET 10 LTS (supported through 14 November 2028) and C# 14.

How to use the next 10 days

Each day is about 3.5 to 4 hours. Spend the first 40 minutes reading the concept block. Spend the next 2 hours saying the answers out loud, timed to 90 seconds, then add the lead point only if they ask. Spend the last 45 minutes writing one design on paper and filling one story from your own projects. Reading this document quietly will not change an interview. Speaking it will.

Who you are in the room

You are not interviewing as a beginner, and you are not interviewing as a principal architect who has never shipped. You have eight years of delivery. The tech lead seat is the person who still designs the Web API, the data access, and the Azure runtime, and who also owns the plan: boundaries, review quality, migration risk, and how the team ships on a bad day.

Open with this, in your own words:

Positioning line

I have eight years building .NET backends. I am moving into a tech lead role where I still own Web API design, EF Core, SQL, and Azure App Service or Functions, and I also own the technical plan: service boundaries, migration risk, code review, and a safe release. I decide with a tradeoff, an operational consequence, and a rollback.

What changes between senior developer and tech lead

They ask	Senior developer answer	Tech lead answer
How does async work?	The state machine, await, and the thread pool.	Where we cancel, where we bound concurrency, and how a slow dependency shows up in the dashboard.
Which pattern?	The name and a class diagram.	The problem it removes, the problem it adds, and the day you would delete it.
Why was production slow?	The missing index.	Who decided to ship, how you rolled back, how the index was reviewed, and what the team now checks every time.
Microservices?	A list of services.	The business capability, the data owner, the failure mode, and why a module inside one deployable is enough until those are stable.
Can you migrate .NET Framework?	Retarget the project.	A slice plan, a compatibility inventory, a shared-database risk, and a date when the old runtime is gone.

Answer shape that scores well

1. First sentence: the decision or the definition. Do not warm up.
2. Second beat: how it behaves in ASP.NET Core, EF Core, SQL, or Azure.
3. Third beat: the tradeoff, with a number if you have one (p95, instance count, lock time, cost).
4. Stop. If they lean in, give the lead point: team rule, review check, or production scar.
5. If you do not know a limit, say which document or metric you would check. Invented Azure numbers lose the room.

Score each practice answer from 0 to 2

- Correct: a working engineer would agree.
- Tradeoff: you named what you give up.
- Consequence: a user, an on-call person, a cost, or a teammate feels it.
- A 6 out of 6 is a hire signal on that question. A correct answer with no consequence still sounds like year four.

The 10 days

Day	Focus	You can do this by the evening
1	C# the panel still tests, at lead depth	Explain memory, LINQ, equality, disposal, and async without hand-waving.
2	SOLID and design patterns	Pick a pattern, and also name the simpler design you would keep.
3	ASP.NET Core Web API	Walk the pipeline, auth, status codes, validation, and a PR review.
4	Entity Framework Core	Explain tracking, N+1, migrations, concurrency, and when you drop to SQL.
5	SQL Server, including database roles	Read a plan, choose an index, explain locking, and lock down the app login.
6	Azure App Service and Functions	Choose a host, a slot strategy, identity, and a poison-message path.
7	Microservices	Draw a boundary, a message, an outbox, and a failure.
8	Legacy .NET to modern .NET	Present a strangler plan a manager can fund.
9	System design and leadership	Two designs on paper and four stories from your own career.
10	Mock interview	90-second answers, rapid fire, and questions you ask them.

Stack to name out loud

Language: C# 14 on .NET 10. API: ASP.NET Core, controller-based for a large team, minimal APIs when the service is small and the team has agreed. Data: EF Core for the aggregate, SQL for the hot query, SQL Server or Azure SQL. Hosting: Azure App Service for the request API, Azure Functions isolated worker for event and schedule work, Azure Service Bus when a message must be completed exactly once from the consumer's point of view. Identity: Microsoft Entra ID and managed identity. Support dates to remember: .NET 8 and .NET 9 reach end of support in November 2026. New work targets .NET 10 LTS through November 2028.

Story bank — fill this before Day 9

Write five short stories from your eight years. Each story needs a system, your decision, a metric, and what the team does now. Leave the sample stories in Day 9 as structure only. Panels can hear a borrowed story.

1. A production incident you led or stabilized. Include the minute you chose rollback or fix-forward.
2. A slow endpoint. Include the query, the plan, and the number after the fix.
3. A migration or upgrade, even a small framework bump. Include what you refused to rewrite.
4. A design you pushed back on. Include the relationship cost and the technical result.
5. A person you grew through review, pairing, or a design doc. Include what they own now.

Day 1 — C# a tech lead can defend

Outcome: you can teach a mid-level developer the language points panels still use to test seniority, and you can connect each one to an API bug you have seen.

Time box (3.5 hours)

- 40 min: concept block below. Say each bullet in one sentence.
- 100 min: Q1 to Q8 out loud. Record the first take. Cut every answer to 90 seconds.
- 40 min: write a one-page explanation of async and CancellationToken as if a teammate shipped a sync-over-async call.
- 30 min: rapid fundamentals at the end of this day. Misses go on tomorrow morning's list.

Concept block

- A class is a reference type. A struct is a value type and lives inline in its container. Copying a large struct copies the data. Boxing wraps a value type in an object and allocates.
- A record is a class with value-based equality, unless it is a record struct. Use a record for a DTO or a message. Use a class when identity matters, such as an entity tracked by EF Core.
- ref passes a variable by reference and the caller must assign it first. out passes by reference and the callee must assign it. in passes a readonly reference and avoids copying a large readonly struct.
- string is immutable. Concatenation in a loop allocates. StringBuilder, or a string.Create / interpolation in modern code for a single result, is the conscious choice. An interpolated string still allocates the result.
- LINQ to Objects is deferred. The query runs when you iterate. A second foreach runs it again. IQueryable carries expression trees so EF Core can become SQL. Materialize with ToListAsync at the boundary.
- async methods return Task or ValueTask and release the thread during the await. async void is for event handlers only. A library method takes a CancellationToken and passes it down.
- ASP.NET Core does not capture a SynchronizationContext. Blocking on .Result is still a thread-pool stall and a deadlock risk in older ASP.NET or UI contexts. In a Web API it shows up as thread starvation under load.
- Equals and GetHashCode stay in sync. A record gives you that. A mutable object used as a dictionary key changes buckets when mutated.
- IDisposable and IAsyncDisposable release unmanaged resources and owned disposables. using and await using define the lifetime. A DI container disposes what it created, for the matching lifetime.
- Nullable reference types are a compile-time contract. Enable them on new projects. A lead treats a warning as a review comment, with a suppression only next to a proven invariant.
- lock uses a private reference object. Do not lock on this, a Type, or a string. ConcurrentDictionary, Channel, and SemaphoreSlim cover the cases lock makes awkward. async code waits with SemaphoreSlim.WaitAsync, because lock cannot cross an await.

- C# features worth naming from the last few versions: records, required members, primary constructors, collection expressions, the field keyword in C# 14 property accessors, and extension blocks for extension members. Name one you use. Skip the feature tour.

Q1. What is the difference between a class, a struct, and a record, and when do you pick each?

Answer. A class has reference identity and lives on the heap. I use it for domain entities, services, and anything EF Core tracks. A struct is a value, copied on assignment, and I keep it small and immutable: a custom id wrapper, a coordinate, a range. A large mutable struct gets copied by mistake and those copies diverge. A record is usually a reference type with value equality, a compact constructor, and a useful ToString. I use records for API contracts, messages, and internal results. A record struct is the value-type version when the data is small and equality should compare contents. EF entities stay classes, because the change tracker depends on reference identity.

> **Say this as a tech lead**
>
> In review I reject a 200-byte struct that is passed through the API layer, and I reject a record used as an EF entity unless the team has a measured reason. The default is class for identity, record for data that is defined by its values.

Q2. Explain ref, out, in, and params without reading a definition.

Answer. ref means the method reads and writes the caller's variable, and the caller initializes it. out means the method must assign the variable before returning; TryParse is the pattern. in means the method receives a readonly reference, which matters for large readonly structs so the call does not copy them. params packs arguments into an array, and in current runtimes params collections can avoid an array in some cases. I almost never design a public API with ref or out. I return a result object or a tuple. Try-pattern methods are the exception the BCL already taught every C# developer.

> **Say this as a tech lead**
>
> A public ref parameter in application code is a design smell in a Web API. It makes ownership unclear and it does not cross an HTTP boundary. I ask the author to return a value.

Q3. Why does this LINQ query hit the database twice, and what is IEnumerable versus IQueryable?

Answer. IQueryable holds an expression tree. EF Core translates it when you enumerate, usually at ToListAsync, CountAsync, or a foreach. If I build IQueryable<Order> query = db.Orders.Where(...) and then both CountAsync and ToListAsync, that is two SQL statements, which is often what I want for a page plus a total. If I assign it to IEnumerable too early, further Where clauses run in memory after the SQL has already returned too many rows. Deferred execution also means a query defined once and enumerated twice runs twice. The boundary rule is: keep IQueryable inside the data method, apply filters, then materialize. Do not return IQueryable from a service that the controller will keep composing, because the SQL shape becomes everybody's problem and the DbContext lifetime gets tangled.

Say this as a tech lead

I want paging, filtering, and the SQL projection in one place. Returning IQueryable from a repository looks flexible and makes query cost invisible. I return a finished list or a paged result.

Q4. Walk through async and await, and tell me where production systems get it wrong.

Answer. An async method returns a Task that represents the operation. At an await of an incomplete task, the method yields. The continuation runs later, normally on a thread-pool thread in ASP.NET Core. The request thread is free to serve other work. The bug I look for is sync-over-async: calling .Result or .Wait on an ASP.NET request. That blocks a thread-pool thread while the operation needs a thread to finish, and under load the pool starves. The second bug is async void outside an event handler, because exceptions skip your try/catch and the caller cannot wait. The third is a missing CancellationToken. When the client disconnects, the handler should stop the EF command and the HTTP call. I pass the token from the controller into the handler, into EF, and into HttpClient. I use Task for public async methods. ValueTask is for a hot path that often completes synchronously, and I do not await a ValueTask more than once.

Say this as a tech lead

A lead standard: every IO method takes a CancellationToken, controllers pass HttpContext.RequestAborted, and a blocked .Result fails review. I also cap outbound parallelism. Starting 500 tasks with Task.WhenAll against one SQL database is a self-inflicted outage.

Q5. How do equality and hash codes work, and why do records change the conversation?

Answer. ReferenceEquals asks if two variables point at the same object. Equals, by default on a class, does the same. Value types compare contents. GetHashCode must match Equals: equal objects must share a hash code, and a hash code must stay stable while the object is in a dictionary or hash set. If I override one, I override the other, and I implement IEquatable<T> on value-like types to avoid boxing. A record synthesizes equality and hash code from its members. That is what I want for a message or a DTO. It is the wrong default for an entity whose identity is the primary key and whose other fields change. Anonymous types and tuples also use value equality, which is useful in tests.

Say this as a tech lead

I ban mutable dictionary keys. I have seen a cache miss that was actually a mutated property inside a key object. Records for contracts, classes with key equality for entities, and a test that locks the contract.

Q6. How should a service dispose resources in ASP.NET Core?

Answer. If a type owns a resource, it implements IDisposable or IAsyncDisposable and releases that resource in Dispose or DisposeAsync. A using declaration keeps the lifetime on the screen. For async cleanup I use await using. In DI, the container disposes the instances it created: at the end of the request for scoped services, and at host shutdown for singletons. DbContext is scoped and disposed by the scope. I do not dispose a singleton inside a request. I do not dispose an instance I received if I do not own it; the factory or the caller does. HttpClient created through IHttpClientFactory is a special case: the factory owns the handlers, and disposing the client per call is safe because the handler pool stays.

Say this as a tech lead

The review comment I write most often is ownership. A new SqlConnection inside a method needs a using. A connection injected from a pool is not disposed by business code. Unclear ownership becomes a leak that shows up only under load.

Q7. What is boxing, and when does it matter on a Web API?

Answer. Boxing allocates an object to hold a value type when it is treated as object or as an interface the value type implements, unless the generic path avoids it. Unboxing casts it back. One allocation is nothing. Boxing inside a tight loop, a logger that formats everything as object, or a non-generic collection is measurable. Generics exist so List<int> does not box each int. LINQ with struct enumerators and Span-based APIs are the runtime pushing the same idea. For a normal CRUD API I care more about chatty SQL than about boxing. I mention boxing when someone stores value types in ArrayList, or when a hot path shows allocations in a trace.

Say this as a tech lead

I ask for a trace before a micro-optimization. If the profiler says the time is in SQL, boxing talk is a distraction. If the profiler says the time is allocations in a parser, then Span, ArrayPool, and fewer boxes are the work.

Q8. How do you make shared state safe, and why is lock the wrong tool across an await?

Answer. Immutable data needs no lock. A request-scoped DbContext is confined to one request, so I do not share it. For a counter, Interlocked is enough. For a cache of independent keys, ConcurrentDictionary is enough. For a critical section that must not run two at a time, I lock on a private object field. The lock statement is reentrant on the same thread and it is released if the block throws. It cannot span an await, because the continuation may run on another thread and Monitor is thread-affine. Across an await I use SemaphoreSlim. A singleton service that touches mutable fields is either immutable, uses a concurrent collection, or serializes access. A static list mutated from requests is a race I expect to see in legacy code.

Say this as a tech lead

I treat a static mutable field in an App Service as a bug the moment we scale to two instances, even if the lock is correct. The lock is per process. The second instance has its own lock and its own memory. Shared state belongs in SQL, Redis, or a queue.

Fundamentals they still ask — short answers

Question	Answer in one or two sentences
abstract class vs interface	An abstract class shares a base implementation and a single inheritance slot. An interface is a contract, and a type can implement many. I start with an interface for a dependency I want to fake. I use an abstract class when subclasses truly share state and behavior.
virtual, override, new	virtual allows a derived class to override. override replaces the virtual method. new hides the base method and breaks polymorphism when the variable is the base type. I almost never use new for methods.
const vs readonly vs static	const is a compile-time constant baked into callers. readonly is set in the constructor and read at runtime. static belongs to the type. A public const in a library is a versioning trap, because callers embed the value.
First vs FirstOrDefault vs Single	First throws if the sequence is empty. FirstOrDefault returns the default. Single throws unless there is exactly one match. On a primary key I use Single or SingleOrDefault so a duplicate fails loudly.
throw vs throw ex	throw keeps the stack trace. throw ex resets it at the new throw. I use throw, or I wrap with the original as InnerException when I add context.
string vs StringBuilder	A string cannot change. Repeated concatenation in a loop copies. StringBuilder mutates a buffer. For a handful of pieces, interpolation is clear and fine.
IEnumerable, ICollection, IList	IEnumerable can be streamed. ICollection adds count and mutation. IList adds indexing. I accept the smallest interface the method needs, and I return a concrete list or a read-only view.
Covariance	out on an interface type parameter makes it covariant: IEnumerable<string> can be used as IEnumerable<object>. in is contravariant, used for consumers such as Action<in T>. I mention this when someone asks why a generic interface will not compile.
Nullable reference types	string? may be null. string should not. The compiler warns. It does not change the CLR. I enable it so reviews talk about null at build time.
partial classes	The compiler merges partial parts. EF migrations and source generators use this. I keep hand-written domain logic out of generated partials so a regeneration does not destroy it.

Evening drill

1. Draw the lifetime of one HTTP request: thread, await on SQL, continuation, response.
2. Explain to an imaginary junior why returning IQueryable from the controller is a support problem.
3. Name one static field in a system you worked on and say where that data should live on two instances.

Traps on Day 1

Do not say value types live on the stack and reference types live on the heap as a complete answer. A struct field inside a class lives with the object. Do not claim ConfigureAwait(false) is required in every ASP.NET Core controller. There is no sync context to return to. Do claim that blocking and missing cancellation still hurt. Do not recite C# 14 features you have not used. One real feature beats a list.

Day 2 — SOLID and design patterns

Outcome: you can answer the classic pattern question a traditional panel expects, and you can show a product panel that you choose the smallest design that keeps the next change local.

Time box (3.5 hours)

- 30 min: SOLID, one Web API example each, spoken.
- 40 min: pattern catalog table. Cover the name, the problem, and the refusal.
- 90 min: Q1 to Q8 aloud.
- 40 min: take one service you know and mark which patterns are actually present. Most codebases need three, not fifteen.

SOLID on a Web API

- Single responsibility: a controller action translates HTTP. A handler performs the use case. EF mapping stays at the data edge. A class that validates, calculates tax, and sends email has three reasons to change.
- Open/closed: a new shipping provider is a new class behind IShippingRate, registered in DI. The checkout handler stays closed. I do not invent a plugin system for two branches of an if.
- Liskov: a subtype honors the contract. An override that throws NotSupportedException for a base method will surprise every caller. Prefer a smaller interface.
- Interface segregation: IOrderQueries and IOrderCommands, or a single focused IClock, beat one IOrderService with forty methods. Callers depend on the slice they use.
- Dependency inversion: the handler depends on an interface it owns. The SQL or HTTP adapter implements it. High-level policy does not reference a concrete SDK type.

> **How to say SOLID in an interview**
>
> Give one example from an API you shipped. Then say which letter you violate on purpose in a small service, because a one-class endpoint is easier to operate than a six-project solution. Judgment is the signal.

Pattern catalog for this role

Learn the spoken answer, not a UML sheet. If they only want the classic name, give the first sentence. If they are senior, give the refusal column.

Pattern	Say this	A .NET shape	Refuse it when
Singleton	One instance, controlled creation.	DI singleton lifetime.	The type holds request data, or you hand-roll a lock around a static instance.
Factory	Creation is a decision worth naming.	A typed HttpClient factory, or a small factory method.	The factory only calls new and has one implementation.
Abstract factory	A family of related objects is created together.	Rare in CRUD APIs. Sometimes a per-tenant infrastructure set.	You have one product, not a family.
Builder	Step-by-step construction of a complex object.	WebApplicationBuilder, or a test-data builder.	The object has two constructor parameters.
Adapter	Make an outside API look like your interface.	A class that wraps a payment SDK.	You wrap a type you own and control.
Decorator	Add behavior around an existing interface.	Caching or timing around IPriceService; middleware is the HTTP form.	The extra behavior is a single line the caller should see.
Facade	One entry hides a messy subsystem.	An application service in front of three legacy calls.	The facade becomes a new god class.
Proxy	A stand-in controls access to the real object.	EF lazy-loading proxy; a caching proxy.	You need the real object and the proxy hides a query.
Strategy	Swap an algorithm behind one interface.	ITaxCalculator per region, chosen by DI or a key.	Two strategies and the choice is a single if.
Observer	Dependents hear a change.	C# events in-process; a message on Service Bus across processes.	You need a guaranteed cross-service workflow. Use a message with a consumer contract.
Command	Represent a request as an object.	A MediatR IRequest, or a queue message.	The command has no value beyond a method call.
Chain of responsibility	Pass a request along handlers until one handles it.	ASP.NET Core middleware and the filter pipeline.	A chain of three classes that always all run. That is a pipeline you should write plainly.
Template method	A base algorithm with replaceable steps.	A base importer with abstract Parse and Save.	The base class forces unused steps. Prefer composition.
Mediator	Objects talk through a mediator.	MediatR, or a single dispatcher you wrote.	Every call is a mediator message, including trivial reads, and stack traces become noise.
Repository	A collection-like boundary over persistence.	A focused data port per aggregate.	The repository only forwards DbSet methods one-to-one.
Unit of Work	One business operation commits as one unit.	DbContext is already a unit of work.	A second unit-of-work wrapper around one DbContext.
Specification	Encapsulate a business predicate.	A testable expression reused by a query.	A specification engine for one Where clause.

Options	Bind configuration to a typed object.	IOptions<T>, IOptionsSnapshot<T>, IOptionsMonitor<T>.	Services read IConfiguration strings from deep inside the domain.
Result	Return success or failure without an exception for expected cases.	A Result<T> for validation and domain rule failures.	You hide defects. A bug still throws.
CQRS	Separate the write model from the read model.	Different handlers, sometimes different stores.	Two models for a form with three fields.
Outbox	Save state and the message in one database transaction.	A row in Outbox, a relay to Service Bus.	You publish to the bus and then save, hoping both work.
Saga	A long business process with compensation.	Durable Functions orchestration, or a process manager.	A single local transaction would do.
Circuit breaker	Stop calling a dependency that is failing.	Resilience pipeline on HttpClient.	You break the circuit on your own database for a single timeout without a policy.
Strangler fig	Replace a legacy system one route at a time.	A facade that routes some URLs to the new API.	A big-bang rewrite with no production traffic on the new path.
Anti-corruption layer	Translate a legacy model so it does not leak.	A mapper at the boundary of the old database or WCF service.	You share the legacy entity project to go faster.

If they name a pattern you rarely use

Have a one-line honest answer. Bluffing a Visitor diagram is worse than a calm scope statement.

- Prototype: clone an object to avoid expensive setup. I copy DTOs explicitly. I rarely need a prototype registry.
- Flyweight: share immutable intrinsic state. String interning is the example I cite. I do not design it into CRUD.
- Bridge: split abstraction from implementation so both vary. I usually just inject an interface.
- Composite: treat a tree of parts like one part. A category tree or a menu. Uncommon in a flat Web API.
- State: behavior changes with state. An order state machine belongs in a named type or a constrained status transition, not a pile of flags.
- Memento: capture and restore state. I use persisted workflow state or an audit table.
- Visitor: add operations over a fixed type hierarchy. I mention it for compilers and document trees, then say I have shipped it rarely.
- Interpreter: a grammar and an evaluator. I have used expression trees via EF, and I do not write interpreters for business rules unless the business truly authors rules.

Q1. Implement a thread-safe Singleton, then tell me why your API does not do that.

Answer. The classic form is a private constructor, a private static readonly instance created by the runtime, and a public static accessor. Static initialization in .NET is thread-safe for that pattern. The lazy form uses Lazy<T> with LazyThreadSafetyMode.ExecutionAndPublication. Double-checked locking is the interview variant I can explain and hope never to maintain. In ASP.NET Core I register the service with AddSingleton.

The container creates one instance for the process, and the type must be thread-safe. I still do not put per-user or per-request data there.

Say this as a tech lead

A handwritten singleton hides the dependency. Tests cannot replace it. A second App Service instance is a second singleton, so it is not a single instance of the business. Cross-instance uniqueness belongs in the database with a unique constraint.

Q2. Factory versus Strategy. Give an example from checkout.

Answer. A factory decides how to build an object. A strategy decides how to do a behavior. At checkout I might have a ShippingLabelFactory that knows which carrier client to construct from configuration. I might have an ITaxCalculator strategy with one implementation per region. The handler depends on ITaxCalculator. Selection can be a keyed service in DI, or a small resolver that maps a country code to an implementation. I keep the resolver dumb. The tax rule lives in the strategy, where it can be tested without HTTP.

Say this as a tech lead

If the team has two calculators and the choice is one if, I leave the if in the handler. I extract a strategy when the third country arrives, or when the rule needs its own tests. Premature strategies scatter a simple policy.

Q3. Where do Decorator, middleware, and filters each belong?

Answer. Middleware sits on the HTTP pipeline and runs for requests that reach it. I use it for correlation ids, exception handling to ProblemDetails, security headers, and request logging. It should be short. Filters sit closer to MVC actions: authorization filters, validation filters, and result filters. A decorator wraps an application interface, such as IPriceService, and adds caching or metrics without the price service knowing. I pick middleware when the concern is HTTP. I pick a filter when it is controller-specific. I pick a decorator when the concern is a use case that might also run from a queue consumer with no HTTP at all.

Say this as a tech lead

Business rules in middleware become invisible. I have seen discount logic in a middleware because it needed the user header. That logic moves into the handler, with a test that does not spin up the host.

Q4. Do you use a generic repository and a unit of work with EF Core?

Answer. DbContext is a unit of work. DbSet is a repository. A second generic repository that exposes Get, Add, and a predicate usually hides none of the complexity and blocks the features I need: Include shape, split queries, projections, and ExecuteUpdate. I use a focused port when the domain must stay ignorant of EF, for example IOrderStore with GetById and Save of an aggregate. The implementation is a thin EF class in the infrastructure project. One SaveChanges per use case commits the unit. I add an explicit transaction when one use case writes through two contexts or needs a SQL lock hint I can explain.

Say this as a tech lead

I will defend a thin repository at an aggregate boundary in a modular solution. I will not defend IRepository<T> as a standard. It is ceremony that makes bad queries harder to see in review.

Q5. Explain CQRS and Mediator as you would use them on a real API.

Answer. CQRS means the write path and the read path can differ. On a small API that is just two handlers and two queries, still on the same database. On a hot read model it can be a projection updated from events, with the understanding that the read side lags. I do not split databases until the read and write need different scaling or different shapes. Mediator is a way to dispatch a command or query object to one handler. MediatR is the common library. It gives a pipeline for logging, validation, and transactions. The cost is indirection: a new developer presses F12 and lands on an interface. I use it when the pipeline behaviors earn their keep. I skip it when the controller can call one application service that is already clear.

Say this as a tech lead

I introduce MediatR with a written rule: commands change state, queries do not, handlers do not call other handlers, and cross-cutting work is a pipeline behavior. Without that rule the mediator becomes a web of hidden calls.

Q6. Options pattern: which interface do you inject?

Answer. I bind a configuration section to a class and validate it on start with ValidateOnStart, so a bad setting fails deployment and not the first request. IOptions<T> is a singleton snapshot read once. IOptionsSnapshot<T> is scoped and sees changes on the next request, which fits a normal Web API that reloads configuration. IOptionsMonitor<T> is for singletons that must observe changes, such as a hosted service. I inject the options type the lifetime requires. I do not pass IConfiguration into the domain. Secrets come from Key Vault through configuration, and the options class exposes only what the feature needs.

Say this as a tech lead

A missing ValidateOnStart is how a slot swap succeeds and the app crashes on the first order. Startup validation is a release check, not a style preference.

Q7. When do you return a Result, and when do you throw?

Answer. I throw for defects and for situations the caller cannot handle: a broken invariant, a null that the type system promised would exist, an IO failure after retries. I return a Result, or a validation problem, for expected business outcomes: invalid input, insufficient stock, a duplicate idempotency key. Validation failures become HTTP 400 with ProblemDetails. A conflict becomes 409. An exception I did not expect becomes 500 and a logged exception with a correlation id. I do not catch Exception at the bottom of every method. One exception handler middleware does that translation.

Say this as a tech lead

I watch for Result types that include a stack trace and an error code for bugs. That hides failures from the hosting model and from crash telemetry. Expected no, unexpected throw.

Q8. How do you decide to add a pattern during a design review?

Answer. I ask what change we are protecting, who feels the pain, and what the pattern costs in navigation and tests. If the change is speculative, I keep the code in one type. If the change has already happened twice, I extract an interface at the seam that varied. I also ask how the failure will look in logs. A pattern that adds three layers and no correlation id has made the system harder to operate. I write the decision in the PR or a short ADR: context, decision, consequences. A tech lead's pattern choice is a team rule, so the next person can repeat it.

> **Say this as a tech lead**
>
> I keep an ADR folder with one page per real decision. In the interview I offer to walk one: why we kept a modular monolith, why we chose Service Bus, or why we dropped a generic repository. That story is more convincing than a pattern list.

A solution shape you can draw

- Api project: controllers or endpoints, auth, middleware, composition root.
- Application project: use cases, ports, validators. No EF reference if you are serious about the boundary.
- Domain project: entities, value objects, domain rules. No ASP.NET reference.
- Infrastructure project: EF Core, HTTP adapters, Service Bus. Implements ports.
- Tests: domain tests with no host; a smaller set of API tests with WebApplicationFactory.

For a small service I collapse Application and Domain, and I say so. Four projects for eight classes is how a lead loses the team.

```
builder.Services.AddScoped<IOrderStore, EfOrderStore>();
builder.Services.AddScoped<CreateOrderHandler>();
builder.Services.AddOptions<PricingOptions>()
    .Bind(builder.Configuration.GetSection("Pricing"))
    .ValidateDataAnnotations()
    .ValidateOnStart();
```

Evening drill

1. Pick three patterns you have shipped. Tell the problem, the shape, and the day you would remove each.
2. Draw the four-project solution. Then cross out a project and justify a simpler layout for a two-person service.
3. Explain generic repository in the classic way, then give the EF Core answer. Some panels want both. Lead with the one that matches the interviewer, and offer the other.

Day 3 — ASP.NET Core Web API

Outcome: you can walk a request from the socket to SQL and back, and you can review another developer's controller the way a tech lead does.

Time box (4 hours)

- 45 min: pipeline order, status codes, and DI lifetimes on paper.
- 100 min: questions out loud.
- 50 min: design a small Orders API on paper: resources, verbs, status codes, auth, errors, paging.
- 45 min: review the sample PR notes at the end and say which comments you would actually write.

Concept block

- The host builds a pipeline. Order matters. A typical API uses exception handling first, then HTTPS redirection, routing, CORS, authentication, authorization, rate limiting, and endpoints. Authentication before authorization, because authorization needs an identity.
- Controllers group HTTP actions. Minimal APIs map handlers directly. I choose controllers when the team is large, filters are already the habit, and the surface is wide. I choose minimal APIs for a small service with a short list of endpoints. I do not mix styles in one service without a reason.
- DI lifetimes: transient is a new instance per resolve, scoped is one per request, singleton is one per process. DbContext is scoped. A singleton must not take a scoped service in its constructor. That captive dependency is either a runtime error or a stale DbContext.
- Model binding turns route, query, header, and body into parameters. [FromBody] is the JSON body. [AsParameters] or explicit sources keep minimal APIs clear. An invalid body is a 400.
- Authentication answers who called. Authorization answers whether that caller may do this. JWT validation checks issuer, audience, lifetime, and signature. Policies combine claims. Roles are a coarse claim. Resource rules, such as owner of this order, need a handler that sees the entity.
- ProblemDetails (RFC 9457, the successor to RFC 7807) is the error body: type, title, status, detail, and extensions such as traceId. Validation errors add an errors dictionary.
- PUT replaces the resource. PATCH updates a subset. POST creates or starts an action. GET is safe. DELETE is idempotent. A client retry of POST needs an idempotency key if a double submit would create two orders.
- IHttpClientFactory plus a typed client plus a resilience handler gives timeouts, retries on idempotent calls, and a circuit breaker. A new HttpClient per call can exhaust sockets. One static HttpClient held forever can ignore DNS changes. The factory is the default.
- Health checks split liveness and readiness. Readiness includes SQL. Liveness does not, or a database blip restarts every instance.
- API versioning is a contract. URL versions are obvious for enterprises. Header versions keep URLs stable and are easier to miss in a gateway. I pick one, document it, and keep an overlap window.

Q1. Walk the request pipeline for a secured POST.

Answer. The request hits Kestrel. Routing selects the endpoint. CORS runs for browser callers. Authentication reads the bearer token and builds a ClaimsPrincipal, or it fails with 401 if the scheme requires a token and the token is invalid. Authorization runs the policy and returns 403 if the identity is known and not allowed. Model binding and validation run. The action or handler executes, opens a scoped DbContext, and awaits SQL. The result is serialized as JSON. If an exception escapes, the exception handler, which was registered early so it wraps the rest, logs it with the correlation id and returns ProblemDetails with 500. Filters, if I use MVC, wrap the action more tightly than middleware: authorization filters, resource filters, action filters, exception filters, and result filters.

> **Say this as a tech lead**
>
> I draw this before I talk about patterns. A lead who cannot order authentication and authorization will mis-design the security review. I also name the correlation id middleware near the start so every log line, including the 500, shares one id.

Q2. Explain DI lifetimes and a captive dependency.

Answer. The container tracks how long an object lives. A transient service is created each time it is requested. A scoped service lives for the HTTP request scope. A singleton lives until the process stops. If a singleton constructor takes a scoped DbContext, the context would be captured for the life of the process: disposed after the first request, or reused across requests and threads. The safe fixes are: make the consumer scoped, or inject IServiceScopeFactory and create a scope at the moment of work, which is what a hosted BackgroundService must do. I also avoid a scoped service that depends on a transient disposable it does not own clearly. The registration is part of the design, so it sits in the composition root, not in the domain.

> **Say this as a tech lead**
>
> On a lead checklist, DbContext is scoped, configuration options that watch reload can be monitor-based inside singletons, and any IHostedService creates a scope per unit of work. I ask candidates to find the captive dependency in a short sample. I expect you to find it too.

Q3. How do you design authentication and authorization for a line-of-business API?

Answer. I use Entra ID to issue tokens and JwtBearer to validate them: authority, audience, and lifetime, with HTTPS metadata. The API does not invent passwords if an identity provider already exists. Coarse access is a policy: can-manage-orders requires a scope or an app role. Fine access is resource-based: the handler loads the order and checks the tenant id and the owner. I keep tenant id in a claim and I also filter in EF with a global query filter, so a missed if in one action does not leak rows. 401 means unauthenticated. 403 means authenticated and refused. I do not return 404 for every forbidden read unless I am deliberately hiding existence, and if I do that I document it.

Say this as a tech lead

I separate the app's runtime identity from the user's identity. The user token decides what the human may do. The managed identity decides what the process may do in Azure SQL or Service Bus. Mixing them into one shared SQL login is how an over-privileged app login appears.

Q4. Which status code do you return for each outcome?

Answer. 200 with a body for a successful read or update that returns a representation. 201 with a Location header when a resource is created. 202 when the work was accepted and will finish later, with a status URL. 204 when success has no body. 400 for validation, with ProblemDetails. 401 for a missing or invalid credential. 403 for a valid credential that may not do this. 404 when the resource does not exist in the caller's scope. 409 for a conflict such as a duplicate idempotency key or a concurrency mismatch. 412 when a precondition such as If-Match failed, if I am using ETags. 429 when the client is over the rate limit, with Retry-After. 500 for a defect on our side. 503 when a dependency is down and the client may retry.

Say this as a tech lead

I publish the error catalog with the API. Clients write logic against status codes, so I do not change 409 into 400 because a junior found it easier. Consistency beats creativity.

Q5. How do you make POST /orders safe under retries?

Answer. The client sends an Idempotency-Key header, a GUID per user action. The API stores the key with the request hash and the response, in the same transaction as the order. A retry with the same key and the same body returns the stored response and does not create a second order. The same key with a different body returns 409. The key has a retention window, for example 24 hours, which matches how long the client retries. Database unique constraints still protect business identity, such as one active cart per user, because an idempotency key does not replace a data rule. GET, PUT, and DELETE should already be safe to retry. I only add the key where the method is not naturally idempotent.

Say this as a tech lead

I treat mobile networks and API gateways as retry machines. If the team cannot explain what happens when the same POST arrives twice, the endpoint is not ready for production.

Q6. How do you validate, version, and paginate?

Answer. Validation lives in one place, usually FluentValidation or data annotations executed before the handler. The handler can assume the shape is valid and still enforce business rules. Errors return the field names the client sent. For versioning I prefer a URL segment for a public enterprise API because gateways, caches, and humans all see it. I run v1 and v2 together until clients move, and I measure calls on v1. For pagination I use limit and a continuation token or a keyset on (createdUtc, id). Offset pagination is acceptable for small admin screens and gets expensive on deep pages because SQL still walks the skipped rows. I cap the page size. Filtering and sorting are an allow-list, never a raw column name from the query string.

Say this as a tech lead

An unbounded GET that returns every order is a production incident waiting for the first large tenant. Paging, a maximum, and a projection that omits heavy columns are acceptance criteria, not polish.

Q7. How do you stop one slow dependency from taking down the API?

Answer. Every outbound call has a timeout shorter than the client's patience. I retry only idempotent operations, with a small jittered backoff, and I do not retry a POST that creates a charge unless the idempotency story is solid. A circuit breaker opens when the failure ratio trips, and it fails fast for a cool-down so the dependency can recover. Bulkhead or concurrency limits stop me from launching unlimited parallel calls. In .NET this belongs in the HTTP resilience pipeline on the typed client. SQL uses a command timeout and an execution strategy for transient errors such as deadlock 1205 or a brief Azure SQL failover. I still surface 503 or a controlled error once the policy is exhausted. I log attempts with the dependency name and the correlation id.

Say this as a tech lead

Retries without a budget multiply load on the thing that is already sick. I would rather fail a request in 2 seconds than have three retries hold threads for 30. The policy is a team default, reviewed like code.

Q8. What do you look for in a Web API pull request?

Answer. I read the contract first: route, verb, status codes, auth policy, and who can see the data. I check cancellation tokens are passed to EF and HttpClient. I look for over-posting: a client must not bind an IsAdmin or TenantId field from the body. I look for sync-over-async, a captive dependency, and a catch that swallows exceptions. I look at the query shape, not only the C#. I check logs do not include tokens, card numbers, or full personal data. I check a test covers the happy path and one failure path that would have cost us money or data. I comment on names when they hide a business rule. I leave style to the formatter.

Say this as a tech lead

My review comments teach a rule. I write: we pass CancellationToken to every IO call so a disconnected client stops the SQL command. The author can apply that rule to the next PR without me.

Sample review, spoken

Imagine this action. Say the comments you would leave.

```
[HttpPost]
public OrderDto Create(OrderDto input) {
    var ctx = new AppDbContext();
    var order = new Order { Id = input.Id, Total = input.Total, IsAdmin = input.IsAdmin };
    ctx.Orders.Add(order);
    ctx.SaveChanges();
    _bus.Publish(new OrderCreated(order.Id));
    return input;
}
```

- The context is constructed by hand, so pooling, configuration, and disposal are bypassed. Inject a scoped store or DbContext.
- SaveChanges is synchronous. Use SaveChangesAsync and accept a CancellationToken.
- The client sets Id, Total, and IsAdmin. The server computes price and sets identity. Bind a create command, not the entity.
- There is no auth policy and no tenant.
- Publish happens after save with no outbox. A crash in between loses the message. A failure after publish with a retry can double-publish if save is repeated.
- Returning the input echoes untrusted data. Return a representation you built.
- No validation, no idempotency, no problem response for a duplicate.

Evening drill

1. On paper, design POST /orders, GET /orders/{id}, GET /orders?limit=, and POST /orders/{id}/cancel. Write status codes and one error body.
2. State the DI lifetime of DbContext, a typed HttpClient, a memory cache, and an IHostedService.
3. Explain 401 versus 403 with a story from a system you know.

Day 4 — Entity Framework Core

Outcome: you can explain what the change tracker is doing, how a slow endpoint becomes a query problem, and how migrations run when two App Service instances start at once.

Time box (3.5 hours)

- 40 min: tracking, loading, and SQL translation.
- 90 min: questions aloud, with one query sketched for N+1 and one for a projection.
- 40 min: write the production migration steps as a runbook.
- 40 min: map one real schema you know onto entities, keys, and a concurrency token.

Concept block

- DbContext is a unit of work and an identity map. By default, a query tracks entities. A second query for the same key returns the tracked instance. SaveChanges writes inserts, updates, and deletes it detected.
- AsNoTracking is the right default for read endpoints. It skips the snapshot cost. You do not update those instances later in the same context.
- Eager loading uses Include. A single SQL query with several collection includes can multiply rows. AsSplitQuery runs separate queries and avoids that explosion. The split queries are not one atomic read.
- Lazy loading pulls related data on property access. It hides N+1 queries. I leave it off unless a specific screen needs it and the team can see the queries in logs.
- Explicit loading is LoadAsync when you already have the parent and sometimes need the children.
- Project to a DTO with Select. The SQL then returns the columns you need. Tracking a full graph to build a list DTO is waste.
- N+1 is one query for parents and one query per parent for children, often from a loop. The fix is a single query with a projection or a controlled include.
- A transaction wraps several SaveChanges calls when they must commit together. The execution strategy retries transient SQL failures and must wrap the transaction in a retry-friendly execution strategy delegate, or the retry cannot re-run your transaction.
- A rowversion concurrency token detects a lost update. SaveChanges throws DbUpdateConcurrencyException. You reload, reapply, or return 409.
- Global query filters apply tenant or soft-delete predicates. IgnoreQueryFilters is an explicit, reviewed escape for admin and migrations.
- ExecuteUpdate and ExecuteDelete write SQL without loading entities. They skip the change tracker and also skip some save interceptors. Use them for bulk set-based work.
- Migrations are source-controlled. Production applies a reviewed SQL script with a dedicated identity. Calling Database.Migrate at startup races when many instances boot together.

Q1. What does the change tracker do, and when do you turn it off?

Answer. When I query with tracking, EF takes a snapshot of the entity. If I change a property, SaveChanges compares current values to the snapshot and writes an UPDATE for the columns that changed. The tracker also guarantees one instance per key in that context, which is useful while I edit an aggregate. Tracking costs CPU and memory, and it can surprise you when a stale tracked instance hides database changes. For queries that only display data, I use AsNoTracking, or AsNoTrackingWithIdentityResolution if I still need one instance per key in a graph. I keep one context per request. I do not cache a context. I do not use one context across threads.

Say this as a tech lead

A read endpoint that tracks every column of every order is a silent tax. I ask the author to project to the DTO. We keep tracking for the command path, where change detection is the feature we want.

Q2. A list endpoint is slow. How do you find an N+1 and fix it?

Answer. I look at the SQL logs or Application Insights dependencies for one request. If I see one select of orders and then a select of lines for each order id, that is N+1, usually from lazy loading or from a loop that queries. I rewrite as one query that projects the page: orders filtered and ordered, Skip/Take or a keyset, Select into the DTO, including a collection only if the UI needs it. If the include of two collections explodes rows, I use AsSplitQuery and I check that the page size is small. I also check that a required filter hits an index. EF cannot make a non-SARGable predicate fast. I measure the request before and after with the same data volume, not with ten rows on a laptop.

Say this as a tech lead

I add a team rule: list endpoints project, they do not return entities, and a PR that adds a loop around a query needs a log sample. I would rather see one explicit SQL statement than a clean-looking loop.

Q3. How do migrations work locally and in production?

Answer. Locally, a developer changes the model and adds a migration, which generates a C# migration and a model snapshot. They review the Up method. EF does not always guess a rename; it may drop and add, and that is data loss. In CI, I generate an idempotent SQL script from the migrations and I review it. In production, a pipeline applies that script with a migration identity that has DDL rights. The application identity does not own the schema. I do not call Migrate() from every instance on startup, because two instances can try to migrate at once and because a bad migration then takes the whole fleet down during boot. A single migrator job, or a pipeline step before traffic shifts, is the controlled path. Backward-compatible steps come first: add a nullable column, deploy code that writes both, backfill, then enforce the constraint.

Say this as a tech lead

The expand/contract pattern is how a lead ships schema changes with rolling instances. The old code and the new code must both run during a slot swap. A migration that renames a column in one step breaks the instances still on the old bits.

Q4. Two users update the same order. What do you do?

Answer. I add a concurrency token, usually a rowversion column mapped as a SQL timestamp and as a concurrency token in EF. The UPDATE includes the original token in the WHERE clause. If zero rows match, someone else wrote first, and EF throws DbUpdateConcurrencyException. The API returns 409 with enough information for the client to reload. I do not catch that exception and blindly retry the save, because I would overwrite the other user's change. For a specific field, a business rule might merge, and that merge is explicit code. A transaction isolation level does not replace a concurrency token. Isolation controls locking during one transaction. The lost update happens when two transactions read, think, and write back in sequence.

Say this as a tech lead

I put the token on aggregates where losing an update costs money or trust: orders, balances, inventory. I do not put a complicated merge on every lookup table. I also keep the write transaction short so locks are not held during an HTTP call to a payment provider.

Q5. How do transactions and the SQL retry strategy fit together?

Answer. SaveChanges uses a transaction. If I need several operations, including raw SQL or two saves, I open a transaction with BeginTransactionAsync, and I commit after the work. Azure SQL can fail transiently: a deadlock, a throttle, a failover. The execution strategy retries those errors. A retry must re-run the whole transaction, so I use the context's execution strategy and put the transaction inside its ExecuteAsync delegate. If I start a user transaction outside that delegate, EF refuses the combination or the retry cannot repeat the work correctly. I make the work idempotent inside the transaction, or I use a transaction id so a retry does not double-insert.

Say this as a tech lead

I never hold a SQL transaction open while I call Service Bus, email, or a card processor. Those calls belong before or after, with an outbox if the message must match the commit. A transaction that waits on HTTP will lock rows and exhaust the pool.

Q6. How do you implement tenant isolation and soft delete in EF?

Answer. Both are global query filters. The tenant filter compares the entity's TenantId to a value taken from the current request, usually a scoped tenant accessor populated from a claim. The soft-delete filter excludes rows where IsDeleted is true. EF adds the predicate to queries. I index (TenantId, ...) so the filter is cheap. IgnoreQueryFilters is rare, audited, and tested, because it bypasses both filters at once; if I need to bypass only one, I model that carefully. Unique indexes include the tenant. A bug that sets the wrong tenant id is a data incident, so the tenant comes from the token, never from the body. For a shared database this is logical isolation. For a high-risk tenant I would give them a separate database and say that clearly.

Say this as a tech lead

A filter is not a security boundary by itself. I still authorize the route, I still deny cross-tenant ids, and I add a test that a token for tenant A cannot read tenant B's key. The test is the control. The filter is the safety net.

Q7. When do you use ExecuteUpdate, raw SQL, or a stored procedure?

Answer. I load and save through the tracker when I am enforcing an aggregate's rules and updating a few rows. I use ExecuteUpdate or ExecuteDelete when I am doing a set operation, such as expiring all unpaid orders older than a date, and I do not want to load them. I use raw SQL or a procedure when the query is a reporting shape EF translates badly, when I need a lock hint I have justified, or when a DBA-owned procedure is the agreed contract. From EF, FromSqlInterpolated parameterizes. I do not concatenate user input into SQL. A stored procedure is a boundary with a cost: it lives outside migrations unless we put it in migrations, and it is easy for the team to forget. I choose it for a hot, stable query, not as the default data layer.

> **Say this as a tech lead**
>
> The lead decision is visibility. If the business rule is in a procedure, the C# review does not see it, so the procedure needs the same PR discipline, a test, and an owner. If the rule is in C#, the SQL must still be visible in logs.

Q8. What EF behaviors do you warn a team about during onboarding?

Answer. Automatic client evaluation is not the old silent behavior; if a predicate cannot translate, I want it to throw, and I keep non-translatable work after materialization on purpose. A cartesian explosion from multiple includes will look like a slow query with a huge payload. A global filter forgotten on a new entity leaks tenant data. A migration that drops a column because a property was renamed will pass CI if nobody reads the SQL. Lazy loading in a loop will pass a unit test with an in-memory stand-in and fail in production. The in-memory provider is not a database. I test query behavior against SQL, at least in one integration environment, for anything with a filter, a transaction, or a constraint.

> **Say this as a tech lead**
>
> I keep a short data-access page in the repo: tracking rules, how to add a migration, how tenant filters work, and the fact that startup migration is forbidden. New joiners should not learn those by causing an incident.

Sketch you should be able to talk through

```
var page = await db.Orders.AsNoTracking()
    .Where(o => o.TenantId == tenantId && o.CreatedUtc < cursor)
    .OrderByDescending(o => o.CreatedUtc)
    .Take(limit)
    .Select(o => new OrderListItem(o.Id, o.Number, o.Status, o.Total))
    .ToListAsync(ct);
```

Point at this and say: no tracking, tenant predicate can use an index, keyset-style cursor, projection, cancellation, and a hard take. If Status is filtered too, the index key should match the filter and the sort, or I show the plan.

Evening drill

1. Draw tracked versus no-tracking for a GET and a PATCH of the same order.

2. Write the expand/contract steps for renaming Order.Total to Order.GrandTotal with zero downtime.
3. Say what you return to the client on DbUpdateConcurrencyException, and what you refuse to do automatically.

Day 5 — SQL Server, query performance, and database roles

Outcome: you can sit with a DBA or a skeptical interviewer and talk about indexes, locking, and the exact permissions the API login has. This is the SQL role conversation: both the query job and the security role.

Time box (3.5 hours)

- 45 min: indexes, SARGability, and a sketch of a plan.
- 70 min: locking, isolation, deadlocks, parameter sniffing.
- 40 min: roles and least privilege, including a migration identity versus a runtime identity.
- 55 min: questions aloud. End by writing GRANT statements you can explain.

Concept block

- A clustered index is the table's row order. One per table. The clustering key rides along in every nonclustered index. A narrow, ever-increasing key such as a bigint identity or a sequential GUID strategy keeps inserts orderly. A random GUID as the clustered key scatters inserts and fragments the leaf.
- A nonclustered index is a separate structure on key columns, with the clustering key attached. INCLUDE adds columns at the leaf so a query can be covered and avoid a key lookup.
- A seek uses the index to jump to the range. A scan reads all or most of the leaf. A scan is not automatically bad on a small table. A seek is not automatically good if it performs a million key lookups.
- SARGable predicates can use an index: column equals value, column greater than value, column in a small list. A function on the column, such as YEAR(CreatedUtc) = 2026, hides the column from the seek. Put the function on the constant, or use a range.
- The optimizer uses statistics. A stale statistic after a big load produces a bad plan. Query Store on Azure SQL keeps plans and runtime stats so you can force a known good plan while you fix the query.
- Parameter sniffing means SQL caches a plan based on the first parameter values. That plan can be wrong for the next shape. Options include rewriting, a local variable with its own tradeoff, OPTIMIZE FOR, or a targeted recompile. I confirm with the plan and Query Store before I sprinkle hints.
- Isolation: READ COMMITTED can block readers and writers on locks. READ COMMITTED SNAPSHOT uses row versions so readers do not block writers for the common case, with statement-level consistency. SNAPSHOT isolation gives transaction-level consistency using versions. Both use the version store. I know which one the database uses before I talk about blocking.
- A deadlock is a cycle of locks. SQL picks a victim, error 1205. The application retries if the operation is safe to retry. The lasting fix is a consistent lock order, shorter transactions, and indexes that remove unnecessary scans and extra locks.
- Keyset pagination filters on the last key seen. Offset pagination asks SQL to count and skip. Deep offsets get slower as the offset grows.
- ACID: atomicity, consistency, isolation, durability. A transaction is the unit. A unique constraint is part of consistency, and I trust it more than a check in C# alone.

Database roles — the security model you should draw

SQL Server has server roles and database roles. The tech lead answer is least privilege for each identity, not a login in sysadmin or db_owner because the app needs to work.

Identity	May do	Must not do
API runtime, managed identity or SQL user	SELECT, INSERT, UPDATE, DELETE, EXECUTE on the procedures it calls	DDL, db_owner, control server, read other databases
Function or worker runtime	Only the tables and procedures that worker uses	Share the API's broader grants by habit
Migration pipeline	DDL on this database during release	Be the login the running site uses every day
Reporting reader	SELECT on a schema or a set of views	INSERT or UPDATE
Human admin	Their own account, audited, privileged through a group	A shared sa password in a wiki
Local developer	A contained dev database with more rights	Those rights copied into production

Built-in database roles you will be asked to name: db_owner can do everything in the database. db_ddladmin can change schema. db_datareader can read all tables. db_datawriter can modify data in all tables. db_denydatawriter blocks writes. These roles are broad. A lead creates a custom role and grants schema-level or object-level permissions.

```
CREATE ROLE app_orders_runtime AUTHORIZATION dbo;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::orders TO app_orders_runtime;
GRANT EXECUTE ON SCHEMA::orders TO app_orders_runtime;
ALTER ROLE app_orders_runtime ADD MEMBER [id-api-prod];  -- managed identity user

CREATE ROLE app_orders_migrator AUTHORIZATION dbo;
GRANT ALTER ON SCHEMA::orders TO app_orders_migrator;
-- used by the pipeline, not by the App Service
```

On Azure SQL, the application uses a managed identity. You create a user from that identity in the database and add it to the runtime role. No password in the connection string. The connection string uses Authentication=Active Directory Managed Identity, or the current Microsoft.Data.SqlClient equivalent your platform standardizes on.

Q1. Clustered index, nonclustered index, and key lookup. Explain them with an orders table.

Answer. Suppose Orders is clustered on Id, an identity. The rows of the table are stored in Id order. A query by Id is a clustered seek. A query by TenantId and CreatedUtc needs a nonclustered index on (TenantId, CreatedUtc) or it scans. If the select list includes Status and Total, and those columns are not in the index, SQL seeks the nonclustered index and then does a key lookup into the clustered index for every row to fetch Status and Total. A few lookups are fine. Lookups for tens of thousands of rows dominate. I INCLUDE Status and Total to cover the query, or I narrow the select list. Every extra included column slows writes and grows the index, so I cover the hot query, not every column in the table.

Say this as a tech lead

I ask for the actual plan on production-like data, or Query Store's plan, before I add an index. An index that helps this report may slow the insert path the business cares about more. I bring one write-heavy example to the interview so the tradeoff is real.

Q2. A stored procedure that used to be fast is slow for one customer. How do you work the problem?

Answer. I capture the statement, the parameters, and the duration from Query Store or from a careful trace. I compare the actual plan to the estimated plan. I look for a scan where a seek should be, a huge mismatch between estimated and actual rows, a key lookup storm, or a spill. I check whether the predicate is SARGable and whether the parameter is atypical, which is parameter sniffing: the plan was compiled for a small tenant and reused for a huge one, or the reverse. I update statistics if a load invalidated them. I fix the query or the index first. I use a hint or a plan guide only when I can name the plan I am pinning and the person who will remove the pin. I also check blocking: a fast query waiting on a lock looks slow in the API and innocent in a local test.

Say this as a tech lead

The API symptom is a latency chart. The SQL symptom is a plan, a wait type, or a lock. I do not scale out the App Service to hide a scan. More instances would run the bad query more often.

Q3. Explain isolation, RCSI, and when readers block writers.

Answer. Under locking READ COMMITTED, a reader takes shared locks and a writer takes exclusive locks. They block each other. READ COMMITTED SNAPSHOT keeps the read committed semantics for a statement and serves the previous version of a row from the version store, so a reader does not take shared locks that block the writer. Writers still block writers on the same row. SNAPSHOT isolation holds a transaction-level view: the transaction sees the database as of its start. It can fail a write with an update conflict if another transaction changed the row. I enable RCSI when the application was suffering reader-writer blocking and the team accepts row versioning. I check tempdb or the accelerated database recovery version store, because versioning is not free. I still use a concurrency token for lost updates. RCSI does not turn two read-modify-write transactions into a safe increment.

Say this as a tech lead

Before I recommend RCSI in an interview, I say I would confirm the current database setting and the write pattern. Turning it on is a database change with a test plan, not a connection-string flag I flip during an outage without understanding the version store.

Q4. What is a deadlock, and what do you change?

Answer. Transaction A locks row 1 and wants row 2. Transaction B locks row 2 and wants row 1. SQL detects the cycle and kills one transaction with error 1205. The victim can retry if the work is idempotent and the execution strategy includes 1205. The design fix is to touch resources in the same order in every transaction, to keep transactions short, and to add an index so a query does not lock a wider range than it needs. I read the deadlock graph: the processes, the resources, the statements. I have seen deadlocks caused by an ORM

updating every column and taking locks in an order the developer did not intend, and by triggers that lock a second table. Retry without a graph is how the same deadlock stays for a year.

Say this as a tech lead

I put a metric on 1205. A retry policy that succeeds still needs a ticket if the rate climbs. The customer saw a retry. The platform saw a design problem.

Q5. How do you paginate a large table?

Answer. Offset pagination, ORDER BY CreatedUtc OFFSET @skip ROWS FETCH NEXT @take ROWS ONLY, is simple and correct for small offsets. For a deep page, SQL reads and discards the skipped rows, and the page shifts if new rows are inserted at the top while the user clicks next. Keyset pagination remembers the last (CreatedUtc, Id) and asks for rows before that pair, with a composite index matching the order. The client sends the cursor, not a page number. I still cap the page size. For an admin UI that must jump to page 400, I talk about the cost openly, or I filter harder so the page number is inside a small set. A covering index makes the page query a seek plus a small range.

Say this as a tech lead

I align the API contract with the SQL. If the product needs stable next-page behavior on a hot table, I do not promise offset page numbers. I design the cursor into the API on day one so we do not break clients later.

Q6. Who should the application connect as, and which role should it have?

Answer. The running API uses a managed identity in Azure, mapped to a user in Azure SQL, added to a custom role such as app_orders_runtime. That role has data rights on the orders schema and execute rights on the procedures it calls. It is not in db_owner, and it is not db_ddladmin. A separate pipeline identity applies migrations and has the DDL grants. Humans use their own admin accounts through group membership, with auditing. Built-in roles like db_datareader plus db_datawriter are a stepping stone I can explain, and they are broader than I want because they cover every table, including audit and payment tables the API should not touch. I grant at schema level when the schema matches the service boundary. I review a new table's grants as part of the migration PR, because a new table with no grant fails safely, and a new table under db_owner fails open.

Say this as a tech lead

If a panel asks whether db_owner is acceptable because migrations run at startup, I say that is two problems: startup migration and an over-privileged runtime. Split the identity, move DDL to the pipeline, and keep the site on the runtime role. That answer is the SQL role answer they are scoring.

Q7. Stored procedure or EF query? Give a decision a team can reuse.

Answer. EF is the default for aggregate commands and ordinary filters, because the model, the migration, and the C# rule stay together. A procedure or a hand-written statement is justified when the plan is critical, the SQL is set-based and ugly in LINQ, or a reporting query must be owned and tuned by someone who lives in Query Store. The procedure is parameterized, lives in source control, and is deployed with the migration.

Rights are EXECUTE, not a blanket datawriter role, when the procedure is the only path and I want to prevent ad hoc updates. I do not move every CRUD action into procedures. That splits the team in half and hides business rules from the people who change them most often.

Say this as a tech lead

I write the decision down. Changing it per developer is how one service ends up with three data-access styles and no owner for performance.

Q8. What is a unique constraint doing that application code cannot?

Answer. Two API instances can both check that an email is free and both insert. The check in C# is not atomic across processes. A unique index is. I use unique constraints for business identity: one idempotency key, one external reference per tenant, one active subscription. I catch the unique violation and return 409. I still validate early for a friendly message, and I accept that the database is the authority under concurrency. A filtered unique index can enforce uniqueness only for active rows, which is the right tool when soft-deleted rows should not block a new value.

Say this as a tech lead

I look for check-then-act in code review. If the bug would create two charges or two users, the fix is a constraint, not a deeper if statement.

Waits and words worth knowing

Term	Plain meaning
CXPACKET / CXCONSUMER	Parallelism coordination. Not automatically a problem. A skewed parallel plan can be.
PAGEIOLATCH	Waiting on data pages from disk or the IO path. Often a scan or a cold cache.
LCK_M_*	Waiting on a lock. Look at the blocker, not only the waiter.
WRITELOG	Waiting on the log flush. Long transactions and chatty commits make it worse.
RESOURCE_SEMAPHORE	A memory grant is waiting. A large sort or hash may need a query change.
SOS_SCHEDULER_YIELD	CPU is busy. The query is burning CPU, or the box is.

You do not need to be the DBA. You need to name the wait, ask for the blocker or the plan, and connect it to a decision in the API.

Evening drill

1. Given WHERE TenantId = @t AND Status = @s ORDER BY CreatedUtc DESC, propose an index and say which column is the seek key and which is included.
2. Explain 1205 to a product manager in four sentences, including what the user felt.
3. Write the two identities for your last system: runtime and migrator. If they were the same account, say what you would split first.

Traps on Day 5

Do not say a clustered index is always the primary key. They often match, and they do not have to. Do not say NOLOCK is a standard fix. It can return torn or missing rows, and a lead reaches for RCSI, indexing, and shorter transactions first. If you have used a read-uncommitted hint, say the specific case and the cost. Do not grant db_owner to the app and call it a role design.

Day 6 — Azure App Service and Azure Functions

Outcome: you can choose a host for a workload, explain how a release avoids downtime, and describe how a function completes, retries, or dead-letters a message.

Time box (3.5 hours)

- 40 min: decision table for App Service, Functions, and a worker.
- 80 min: questions aloud.
- 45 min: draw a release: slot, health, swap, connection string, managed identity.
- 45 min: draw one Service Bus function: peek lock, work, complete, abandon, dead-letter.

Where the code runs

Host	Fits	Watch
App Service	The HTTP API, always on, predictable latency	Scale-out means many instances. In-memory state is not shared. Use slots for release.
Functions, isolated worker	Events, schedules, file arrival, short work behind a queue	Cold start on Consumption. A long or chatty workload belongs on Premium or a dedicated plan.
Durable Functions	A workflow with waits, fan-out, and compensation	Orchestrations stay deterministic. Real work goes in activities.
Container Apps or AKS	Custom runtime, sidecars, or a platform standard you already operate	You are taking on a platform. I do not start there for one Web API.
WebJobs or a hosted BackgroundService	Work that must live next to the API and is simple	It scales with the API even when the queue is the thing that is busy.

Isolated worker

New functions target the isolated worker model. Your function code runs in a separate process from the host. That is the model that moves forward with .NET 10. The older in-process model, where the function shared the host's CLR, is the legacy path. In an interview, say isolated worker, say why cold start exists, and say which plan you would buy for the latency you need.

Q1. App Service or Functions for an order API that also sends email?

Answer. The HTTP API goes on App Service: it has steady traffic, it needs a stable latency, and it owns the request. The email goes on a queue. A function triggered by Service Bus sends the email and completes the message. If email is slow or the provider is down, the API has already committed the order and returned 201. The function retries with the Service Bus delivery count. I do not send email inside the POST, and I do not put the whole CRUD API on a Consumption function if the product expects hot latency all day. A function is a fine host for a small HTTP API with spiky traffic if cold start is acceptable or the plan keeps instances warm.

Say this as a tech lead

I split by scaling and failure, not by technology fashion. The API and the email worker scale for different reasons. If they share a process, a mail-provider outage and a retry loop can starve the API threads.

Q2. How do deployment slots give you a safer release?

Answer. A staging slot runs the new bits with production-like settings. I warm it up, I hit the health endpoint, and I run a smoke check. Swap exchanges the slots. The old bits remain on the staging slot for a swap-back. Connection strings and app settings can be sticky to a slot, and I am deliberate about that: a staging slot must not point at a destructive migration by accident, and it must not keep a staging secret after swap unless I marked it sticky on purpose. Always On stays on for the API so the process is there. Health checks on the App Service should hit readiness, including a lightweight dependency check, so traffic does not land on a bad instance. Migrations run as a pipeline step before traffic moves, using the expand/contract rule from Day 4.

Say this as a tech lead

A swap is not a migration strategy by itself. If the new code requires a column the old code cannot read, swap-back is a lie. I only call a release reversible when the previous bits still work against the current schema.

Q3. How does the API get a secret and reach Azure SQL without a password?

Answer. The App Service has a system-assigned or user-assigned managed identity. I grant that identity access to Key Vault secrets it needs, and I reference those secrets from app settings so they appear as configuration. For Azure SQL I create a user for that identity and add it to the runtime database role. The app uses Azure identity credentials in the SQL client. There is no SQL password and no secret in source control. Locally, developers use their own Entra login or a dev database with a user secret stored outside git. Key rotation becomes an identity story, not a password-mailing story. I still validate options on startup so a missing vault reference fails the slot before swap.

Say this as a tech lead

I can draw three identities: the user's token, the API's managed identity, and the pipeline's migration identity. If I cannot name all three, the design is not finished.

Q4. What is a cold start, and which plan do you choose?

Answer. On the Consumption plan, Azure can scale to zero. The next invocation loads the host and your isolated worker, and that delay is the cold start. Premium and dedicated plans keep instances warm, with a higher floor cost. I choose Consumption for low-volume jobs, scheduled cleanup, and work that can wait a second. I choose Premium or an App Service plan when a user is waiting, or when the function must hold a connection and run longer with predictable throughput. I also watch the timeout. A function that processes a large file in one invocation will hit the plan limit. I split the file into messages or I move that workload to a host with a duration that matches the job.

Say this as a tech lead

I put the plan in the design doc with the latency target. Choosing Consumption because it is cheap, then discovering checkout waits on a cold start, is a lead miss. Cost is part of the decision, and so is the user waiting.

Q5. Walk a Service Bus trigger from delivery to completion.

Answer. Service Bus delivers a message under a peek lock. The lock is a lease, not a deletion. The function runs. If it finishes, it completes the message and Service Bus removes it. If it throws, the message is abandoned and becomes visible again after the lock, up to MaxDeliveryCount. After that it is dead-lettered. I inspect the dead-letter reason, fix the poison or the bug, and have a replay path. I renew the lock, or I keep the work shorter than the lock, so a long job is not processed twice by another worker who received it after the lock expired. The handler is idempotent: if the same message arrives twice, the database unique key or a processed-message table stops a double charge. I set prefetch carefully. A huge prefetch on a slow handler holds messages that another instance could have finished.

Say this as a tech lead

I can describe the poison message from my own system, or a realistic one: a payload that always throws, burns the delivery count, and sits on the dead-letter queue while the rest of the queue stays healthy. The lead's job is the alert on dead-letter depth and the replay runbook, not only the happy complete.

Q6. What belongs in a Durable Functions orchestration?

Answer. An orchestration coordinates. It can wait for events, fan out activities, and run compensations. It must be deterministic: no direct IO, no DateTime.Now, no random GUID inside the orchestrator. Those go in activities, and the orchestrator calls them through the context. State is checkpointed, so a replay must make the same decisions. I use Durable Functions when the workflow is long, has human waits, or needs a clear compensation owner, such as reserve inventory, take payment, and release inventory if payment fails. I use a single message and a database row when the workflow is one step. I do not hide a simple retry policy inside an orchestration.

Say this as a tech lead

Determinism is the question that separates someone who read the feature from someone who has operated it. I mention replay explicitly. If the interviewer has not used Durable Functions, I still show I know when a row and a queue are enough.

Q7. What breaks when you scale from one instance to ten?

Answer. Anything in static fields or in-memory cache becomes ten copies, possibly inconsistent. A background timer inside the API runs ten times unless I use a distributed lock or move it to a single scheduled function. A file on local disk is not shared. Session affinity on App Service hides the mistake and then sticks users to dead instances, so a stateless API turns affinity off. SQL connections scale with instances, so ten instances with a fat pool can exhaust Azure SQL. I set the pool size with the max instances in mind. A singleton lock still only locks one process. Distributed coordination uses SQL, Service Bus sessions, or a purpose-built store.

Say this as a tech lead

I ask every design: what happens on the second instance? If the answer is we only run one, I ask what happens when the platform restarts that one. Single instance is a availability decision, and I want it on paper.

Q8. Production is slow right now. What do you open first?

Answer. I start from the user symptom and the correlation id, then the request chart: rate, latency, errors, and dependency time. If SQL time dominates, I go to Query Store and blocking, not to a code deploy. If an outbound HTTP dependency dominates, I check timeout, retry storms, and the circuit breaker. If CPU is high on the App Service and SQL is fine, I look for a hot loop, bad cache, or thread-pool starvation from blocking calls. I check recent deploys and a slot swap time. I decide rollback versus fix-forward with a clock: if the last deploy lines up and rollback is safe, I swap back first and diagnose second. I communicate a one-line status while I do it. Afterward I write the cause, the customer impact, and the guard that would have caught it.

Say this as a tech lead

The panel is listening for sequence and calm. Metrics, a reversible action, then root cause. A lead who opens Visual Studio and reads code for twenty minutes while the slot can be swapped back has the wrong order.

Settings worth naming

- App Service: Always On for the API, HTTPS only, TLS policy, health check path, ARR affinity off for a stateless API, one identity per app.
- Functions: the plan, the timeout, the concurrency per instance, Service Bus MaxDeliveryCount, dead-letter alerts, and a retry policy that matches idempotency.
- Both: structured logs, correlation id, no secrets in logs, availability test from outside the region, and a dashboard that shows dependency duration separately from your own CPU.

Evening drill

1. Draw the order API on App Service and the email function on Service Bus. Mark the transaction boundary.
2. Say what sticky setting means during a swap, using a connection string as the example.
3. Explain peek lock in under a minute, then say what you do when the dead-letter queue grows.

Day 7 — Microservices, as a person who has to operate them

Outcome: you can defend a service boundary, a message contract, and a failure, and you can also defend keeping one deployable when the team and the domain are not ready to split.

Time box (4 hours)

- 30 min: vocabulary — capability, data owner, sync, async, consistency.
- 100 min: questions aloud.
- 60 min: draw an order system with two services and one message. Include the outbox.
- 50 min: failure game. Kill SQL, kill the bus, deliver a message twice, deliver it late. Say what the user sees.

Concept block

- A microservice is a deployable with its own data and a business capability, owned by a team that can release it. A folder in a solution is a module. A database schema with a separate login is a step toward a boundary. A separate process with a shared database and shared tables is a distributed monolith.
- Start from a modular monolith when one team owns the domain and the boundaries are still moving. Extract a service when a part needs its own scale, its own release train, or a different availability target.
- Each service writes its own data. Another service gets that data through an API or a message, or it keeps its own copy that it accepts may lag.
- Synchronous HTTP is for a query the user is waiting on, when the caller can tolerate the dependency. Asynchronous messaging is for work that can finish after the response, and for fan-out.
- Azure Service Bus is for commands and business events that need locks, dead-letter, sessions, and a consumer. Event Grid is for event distribution and reacting to platform events. Event Hubs is a high-volume stream, closer to telemetry and ingestion than to one business command.
- The outbox pattern writes the business row and the message row in one transaction. A relay publishes the message and marks it sent. Consumers are idempotent.
- A saga is a long-running process with a forward path and a compensation path. Orchestration puts the policy in one process manager. Choreography lets each service react to events. Orchestration is easier to see and to change. Choreography couples services less and becomes harder to trace.
- Idempotency means doing the message twice has the same business result as doing it once. At-least-once delivery is the realistic Service Bus contract. Exactly-once processing is your deduplication, not a checkbox on the bus.
- Observability is logs, metrics, and traces with one correlation id crossing HTTP and the bus. A tech lead budgets for this in the first release.

Q1. When would you split a service, and when would you keep a modular monolith?

Answer. I keep one deployable while one team is learning the domain, the transactions are local, and we can still draw modules with separate projects and separate schemas. I split when a capability has a different scaling profile, a different compliance boundary, or a team that must release without waiting for the rest.

Orders and catalog often split because catalog reads dwarf order writes, or because different teams own them. I do not split because the diagram looks modern. Each split adds a network failure mode, a contract, a dashboard, and an on-call question. If I cannot name the data owner and the failure, the split is early.

Say this as a tech lead

In the interview I draw modules first, then one dotted line where I would cut, and I say what has to be true before I cut: an independent schema, a contract test, and a team that can run the new service next week. That sequence sounds like a lead. A page of boxes on day one sounds like a diagram.

Q2. Why is a database per service the hard part, and how do you query across them?

Answer. The hard part is the report that used to join Orders and Customers. After the split, that join is a distributed query. I handle it in one of three ways. The UI calls both services and composes, which fits a small screen. One service stores the few customer fields it needs, updated by events, which fits a hot path and accepts lag. A reporting store ingests both streams and serves the warehouse-style question, which fits analytics. I do not give the order service a SQL login on the customer tables. That shortcut recreates the coupling and hides it from the API review. Cross-service transactions that need both commits to succeed become a saga, not a TransactionScope over two databases.

Say this as a tech lead

I say the reporting question out loud in design reviews, because it is the reason teams sneak shared databases back in. If I have an answer for the report, the boundary will hold.

Q3. Explain the outbox. What fails if you publish after SaveChanges?

Answer. If I save the order and then publish, a crash or a bus outage between the two leaves a committed order and no message. The email never sends, and the inventory service never hears about it. If I publish and then save, a failed save leaves a message for an order that does not exist. The outbox inserts the order and an outbox row in one SQL transaction. A relay reads unpublished rows, sends them, and marks them published. The relay can send twice if it crashes after send and before the mark, so the consumer deduplicates on the message id. I poll the outbox or use a transaction log reader if the volume requires it. Polling is the version I can explain and operate.

Say this as a tech lead

I would rather have a visible unpublished row I can alert on than a message that exists only in memory. The outbox table is an operational object. It has a count, an age, and an owner.

Q4. Design a saga for order, payment, and inventory.

Answer. The order service accepts the command and writes an order in a Pending state. An orchestrator, which could be Durable Functions or a process manager in the order service, asks inventory to reserve, then asks payment to capture. If payment fails, it sends a release to inventory and marks the order Failed. If the release fails, the saga stays in a compensating state and alerts a human. Inventory and payment each enforce their own local transaction and their own idempotency key. There is no distributed lock across the three databases. Timeouts are explicit: a reservation expires if the saga never completes. I choose orchestration

here because the compensation policy is a business rule I want in one place, testable, and visible to support. Choreography would hide that policy across three consumers.

Say this as a tech lead

I name the stuck states. A lead design includes Pending too long, Reserve succeeded and Payment unknown, and Compensation failed. Those states need a screen or a query, not only a log line.

Q5. What does idempotent consumption look like in SQL?

Answer. The message has a stable id. The consumer inserts that id into a ProcessedMessage table with a unique primary key in the same transaction as the business write. If the insert hits a duplicate key, this message was already applied, and the consumer completes it without doing the work again. For a create, the business unique key can be the idempotency key itself. For an increment, I store the message id so I do not add the quantity twice. I complete the bus message only after the transaction commits. If I complete first and then crash, the work is lost. If I commit first and then crash before complete, the message redelivers and the unique key saves me.

Say this as a tech lead

I test this. A unit test that calls the handler twice is the cheapest proof. I have seen teams say the bus is exactly once and skip the unique key. One redelivery then double-shipped an order.

Q6. How do you version an event?

Answer. I treat the event as a contract. I include a version, a message id, a correlation id, an occurred time, and a payload that is tolerant of new fields. Adding an optional field is a compatible change. Renaming or changing a meaning is a new event type or a new version, and I publish both until consumers move. I do not share a C# class library of entities as the contract across services, because a rebuild then couples their releases. A small contract package or a published schema is enough. Consumers ignore fields they do not understand. They do not throw on an unknown optional field and dead-letter the queue.

Say this as a tech lead

A shared EntityFramework model project referenced by two services is a distributed monolith with extra steps. I say that clearly if they ask how teams share data models.

Q7. What do you log, measure, and trace across services?

Answer. Every request and every message carries a correlation id, and if a message causes another message, a causation id. Logs are structured JSON: correlation id, service name, message id, tenant, and a duration. I do not log the full payload when it contains personal data. Metrics: request rate, latency, error rate, dependency latency, queue depth, dead-letter depth, outbox age, and saga state counts. Traces connect the HTTP call to SQL and to the consumer using OpenTelemetry. I alert on symptoms users feel and on queues that will become those symptoms soon. I keep one dashboard per service and one picture of the business flow.

Say this as a tech lead

I budget tracing in the first milestone. Adding correlation ids after five services are in production is a project. A lead who ships the split without the dashboard has created a system the team cannot be on call for.

Q8. A downstream service is down. What does the user see?

Answer. If the call was synchronous and required, the user sees a controlled 503 after the timeout and the circuit breaker, and we do not hold threads for a minute. If the work was asynchronous, the user sees 202 Accepted and a status they can poll, and the message waits on the bus or in the outbox. I decide that classification when we design the endpoint, not during the incident. I also decide the backlog behavior: if inventory is down, do we keep accepting orders into Pending, or do we stop at the edge? That is a business decision I record. Automatic retry with exponential backoff and a dead-letter path keeps the rest of the platform moving.

Say this as a tech lead

I bring the product owner into the accept-versus-reject choice before go-live. Technology can implement either. The wrong default is an accident.

Draw this from memory

1. Client calls Order API. API writes Order and Outbox, commits, returns 201.
2. Relay publishes OrderPlaced to Service Bus and marks the outbox row.
3. Email function and Inventory consumer both receive. Each deduplicates.
4. Inventory failure retries, then dead-letters. Order stays in a state that support can see.
5. Correlation id is on the HTTP log, the outbox row, and both consumers.

Evening drill

1. Name two capabilities in your current system that are modules, and one that could be a service. Say why the third is different.
2. Explain at-least-once delivery and the unique key in under two minutes.
3. List the stuck states for a payment saga and who gets paged.

Day 8 — .NET Framework to modern .NET

Outcome: you can present a migration a manager can fund and an engineer can start on Monday. The target runtime is .NET 10, because .NET 8 support ends in November 2026.

Time box (3.5 hours)

- 40 min: inventory list and the strangler picture.
- 90 min: questions aloud, using a real system if you have one.
- 50 min: write a six-step plan with risks, owners, and what you will not rewrite.
- 30 min: EF6 to EF Core gaps, spoken as a table.

The lead's opening line

I would not rewrite the system in a branch for nine months. I would inventory the blockers, put a facade in front of the surface we must keep stable, move one vertical slice to .NET 10, run both, and delete the old slice only when the new one has the same logs, the same auth, and a rollback. The goal is an empty Framework footprint on a date, not a heroic cutover weekend.

Concept block

- Modern .NET is cross-platform, uses the ASP.NET Core pipeline, the SDK-style project, PackageReference, and appsettings plus environment variables plus Key Vault. .NET Framework is Windows-first, with System.Web, Global.asax, web.config, OWIN in some apps, and often EF6 or ADO.NET.
- The .NET Upgrade Assistant and the API analyzer find APIs that do not exist on .NET 10. They do not find behavioral changes or a bad module boundary. The lead still reads the report.
- A class library can multi-target net48 and net10 for a bridge period if the code stays on the shared surface. That lets old and new hosts share a pure domain library. It is a temporary state with an end date.
- ASP.NET to ASP.NET Core replaces System.Web. HttpContext is different. Session, cache, and authentication modules move to middleware and services. Web Forms does not come along. MVC views can move with work. Web API controllers move more naturally.
- WCF does not ship as a full equivalent. CoreWCF keeps some SOAP services alive on modern .NET. A public contract that you can change should become REST or gRPC. I do not promise every WCF binding will appear.
- EF6 uses an older model, sometimes EDMX. EF Core uses code-based models, different lazy loading, different interceptors, and migrations you own in code. Some queries translate differently. You test the expensive queries against SQL.
- Authentication often moves from Windows auth or OWIN cookies to Entra ID and JWT for an API, or to ASP.NET Core cookie auth for a site. I do not leave anonymous endpoints by accident during the port.
- System.Drawing, COM, the registry, Event Log assumptions, Windows services, and impersonation are the usual blockers. I list them in week one.

- The strangler fig puts a routing layer in front. Some routes go to the new host. The rest stay. IIS, YARP, or a gateway can do that routing. Data compatibility is the real constraint.

Q1. Give me your migration plan.

Answer. Week zero is an inventory: projects, target framework, NuGet packages, System.Web usage, WCF, EF6 or EDMX, auth type, Windows-only APIs, scheduled jobs, and how we deploy. I run the upgrade analyzer and I sit with someone who knows the ugly paths. I pick .NET 10 as the target and I freeze new features that deepen Framework-only APIs. I add characterization tests around the slices we will move, at the HTTP boundary if unit tests do not exist. I move a pure class library first, then one vertical API slice, behind the existing URL via a reverse proxy. I ship that slice to production traffic, small, with the old path still available. I repeat. Schema changes use expand/contract so both runtimes can work. I remove a Framework project only when nothing calls it. I report percent of traffic on the new runtime, not percent of projects converted.

> **Say this as a tech lead**
>
> Traffic percent is the metric I show a steering group. Converted projects can be the easy libraries while the revenue path is still on Framework. I have seen a status deck that said 70 percent migrated because seven of ten projects were class libraries nobody deployed alone.

Q2. What usually blocks a Web API migration?

Answer. HTTP modules and handlers that quietly do auth, logging, or tenant resolution. Those become middleware, and if I miss one, the new host is missing a control. WCF clients generated from old references. Binary serialization and Remoting, which should become an explicit contract. Configuration in ConfigurationManager and web.config transforms, which become the options pattern. Static HttpContext.Current, which has no place in ASP.NET Core; the context is passed or accessed through IHttpContextAccessor sparingly. App-pool identity assumptions, which become a managed identity. Blocking synchronous IO that happened to work at low volume. And packages that were never updated past a .NET Framework build. I classify each blocker as replace, wrap for now, or stop the slice.

> **Say this as a tech lead**
>
> I publish the blocker list with an owner and a decision. A migration without that list becomes a long branch. The list is also how I say no to a rewrite of the reporting module that is not on the critical path.

Q3. How do you move EF6 to EF Core without a surprise in production?

Answer. I treat it as a new data layer, not a namespace rename. I recreate the model in code, map keys, required relationships, delete behaviors, and column types explicitly, and I compare the schema EF Core wants with the schema that exists. I do not let it create a parallel database by accident. I port the hot queries first and run them against a copy of production statistics or a realistic dataset, because translation differences change plans. Lazy loading is opt-in and different; I prefer explicit loads and projections. EDMX does not come forward. I keep EF6 in the old process and EF Core in the new process during the strangler, both pointed at the same database only with strict rules: no silent model drift, and migrations owned by one side. Spatial types, table-per-type inheritance, and some complex mappings need an explicit check. I test concurrency tokens and transactions, not only CRUD.

Say this as a tech lead

One owner of the schema during the overlap is non-negotiable. Two migration histories on one database is how a Friday deploy drops a column the old app still reads.

Q4. How do you migrate authentication safely?

Answer. I write down the current mechanism: Windows integrated, forms cookie, OWIN JWT, custom headers, or certificates. I map every audience: browser, partner, and internal job. For a new API I prefer Entra ID and bearer tokens, with policies that match today's roles. I run both authenticators during overlap if clients cannot all move, and I log which scheme succeeded. I test 401 and 403 on every moved route, plus one cross-tenant case. I do not trust a ported [Authorize] until I see a test fail closed. Secrets in web.config move to Key Vault references. Certificates get an owner and an expiry alert. A Windows-identity flow that relied on impersonation to SQL is a design change: the user's identity stays in the token, and the app's managed identity connects to SQL, with authorization enforced in the API.

Q5. What do you test, given the old system has few tests?

Answer. I start with characterization tests on the HTTP contract of the slice I am moving: status code, important headers, and the JSON shape for the main success and the main failure. I add golden tests around billing calculations and any code that has caused an incident. I add an integration test that hits SQL for the migrated query. I do not stop the migration to unit-test the entire legacy solution. I do not accept a rewrite of a slice with no before-and-after contract check. For the proxy, I test routing rules: this path goes to the new host, that path stays, and a bad route fails in a way we can see. Load on the migrated slice is a test, because sync-over-async and missing indexes show up there.

Say this as a tech lead

I sequence tests by money and embarrassment. The payment path gets a characterization test before the admin dropdown does. That is a lead tradeoff I will say in the interview.

Q6. How do you estimate, and what do you refuse to rewrite?

Answer. I estimate the inventory and the first slice, not the whole estate. The first slice teaches us the real cost of auth, EF, and deployment. After it is in production I estimate the rest in slices, with a range. I refuse to rewrite code we do not understand, code with no contract, and code the business is about to replace. I wrap those and put an expiry on the wrap. I also refuse a second feature program that lands only on Framework while we migrate. New work goes to the new runtime unless there is a dated exception. I keep a risk register: shared database, unknown module, key-person knowledge, package with no modern build, and a rollback that requires schema compatibility.

Say this as a tech lead

A single number for an 80-project solution is fiction. I give a funded first slice, a risk register, and a cadence. That is the estimate a tech lead can defend.

Q7. How does rollback work six weeks into a strangler?

Answer. Routing rollback sends the slice's traffic to the old host. That only works if the old host can still read and write the current schema, which is why we expanded the schema and did not rename or drop in the same release. Data written by the new service must be readable by the old service, or I have a compensating job I have rehearsed. Configuration and feature flags can disable a new behavior without a deploy. I do not need to roll back the class libraries that both hosts share if they stayed compatible. I rehearse the route switch in a lower environment. If I cannot describe rollback, the slice is too big.

> **Say this as a tech lead**
>
> I put rollback on the release checklist next to the migration script. The person on call should not invent it at midnight.

Q8. Which .NET features do you adopt during the move, and which do you postpone?

Answer. I adopt the SDK-style project, nullable reference types on new code, generic hosting, the options pattern, structured logging, and EF Core migrations owned by the pipeline. I adopt managed identity if the app is moving to Azure as part of this work. I postpone a microservices split, a new UI framework, and a domain rewrite. The migration's job is to get onto a supported runtime with the same business behavior and a safer deploy. A cleaner module boundary inside the new host is welcome when it falls out of the slice. A new architecture program disguised as a migration is how the date slips and Framework stays in production past support.

> **Say this as a tech lead**
>
> I say this to stakeholders in one sentence: supported runtime first, structural improvement only where the slice already has to change. That sentence has saved more migrations than a pattern catalog.

Blockers you should be able to classify

Blocker	A lead's classification
System.Web HttpModule	Replace with middleware in the slice. Test the behavior it used to own.
WCF SOAP the partner still calls	CoreWCF as a bridge, or keep that host until the partner can move. Do not block the rest of the API.
EDMX	Replace with a code model for the migrated slice. Compare schema. Do not auto-drop.
Web Forms page	Leave it, or rebuild that page as a separate decision. It is not a class-library port.
AppSettings in web.config	Options classes, environment overrides, Key Vault for secrets.
Windows service	Worker Service on .NET 10, or a Function if it is a schedule or a queue.
COM or registry	Wrap and isolate. It may force that component to stay on Windows even when the API moves.
Shared SQL login as db_owner	Split runtime and migration identities as part of the first production slice.

Evening drill

1. Write a one-page plan for a system you know, even if you change the names. Include the first slice and the metric.
2. Name three APIs or libraries that would not port cleanly, and classify each.
3. Explain expand/contract plus route rollback in under two minutes.

Day 9 — System design and the lead conversation

Outcome: you can run a 25-minute design without jumping to Kubernetes, and you can tell four stories from your own eight years using a structure a panel can score.

Time box (4 hours)

- 20 min: memorize the design spine below. It is the outline you write on the board in the first three minutes.
- 90 min: Design A and Design B, spoken, with a timer. Stop at 20 minutes and summarize.
- 70 min: write your four stories. Use the sample only as a skeleton. Replace every fact.
- 40 min: behavioral questions aloud, 2 minutes each.

The design spine

1. Clarify the user, the success metric, the scale you are assuming, and what is out of scope. State assumptions and let them correct you.
2. Name the API: resources, verbs, sync versus async, status codes.
3. Name the data: the aggregate, the key, the constraint, the index that matches the hot query.
4. Name consistency: what is in one SQL transaction, what is a message, what may lag.
5. Name failure: timeout, retry, duplicate, poison message, and what the user sees.
6. Name operation: logs, the one dashboard, deploy and rollback, identities and SQL roles.
7. Name the tradeoff you refused, and the first slice you would ship.

> **Scale numbers**
> If they do not give numbers, assume some and say them. For example: 50 requests a second peak, 2 million orders a year, p95 under 300 milliseconds for the read, email within a minute. Invite correction. Design changes when the number changes. A lead who designs for a million events a second without being asked will overbuild and run out of time.

Design A — Order API

Prompt: design the backend for creating an order, viewing it, and emailing a confirmation. Partners retry. Two customers must not lose updates. You are the tech lead.

- POST /orders accepts a command and an Idempotency-Key. The server prices the order. The client does not send IsPriority or Total as authority. Response is 201 with a body and Location, or 200 with the original body if the key repeats.
- The write is one SQL transaction: order, lines, idempotency row, outbox row. Unique constraint on the idempotency key and on any business key. rowversion on the order. PATCH returns 409 on a conflict.

- GET /orders/{id} is a no-tracking projection authorized by tenant and policy. List uses a keyset and an index on tenant, created time, and id.
- A relay publishes OrderConfirmed. A function sends email. The consumer deduplicates. The user does not wait on the mail provider. Status is visible on the order.
- Host the API on App Service with slots, managed identity, and a runtime SQL role. The function has its own identity and only the rights it needs. Migration identity is separate.
- Timeouts and a resilience policy on any sync dependency such as a tax service. If tax is down, decide with the business: reject the order, or accept PendingTax. Say which you recommend and why.
- Ship the synchronous create and the outbox before you split a pricing microservice. Pricing can be a module.

Close with: the risk I would watch in week one is a duplicate email from redelivery, so the unique message id ships in the first increment, not later.

Design B — Nightly file of a million rows

Prompt: a partner drops a CSV in storage every night. You must validate, load, and report failures. The HTTP API must stay responsive.

- A blob trigger or an Event Grid event starts a function that only records the file and enqueues a control message. It does not parse a million rows inside one invocation.
- A splitter reads the file in chunks and enqueues chunk messages with a file id and a byte range or row range. Workers on a Premium plan or a dedicated plan upsert rows.
- Each chunk commits with a row-range idempotency key so a retry does not double-insert. Failures of a row go to an error table with the line number and the reason. The file has a state: Received, Processing, CompletedWithErrors, Failed.
- SQL uses bulk copy or a staged table and a set-based merge for speed. EF SaveChanges per row will not make the window. Say that plainly.
- The API serves status and error pages from SQL. It does not handle the file on a request thread.
- An alert fires if the file has not reached a terminal state by a business deadline. A poison chunk dead-letters without stopping the other chunks, and the file state shows the gap.
- Identity: the worker's managed identity can read the container and write the staging schema. It cannot alter the schema.

Close with: I would prove chunk retry on a 10 MB file before I talk about scale, and I would show the error CSV the partner can fetch.

Design C — if they ask multi-tenant

- Default: one database, TenantId on every tenant-owned row, a global query filter, a unique index that includes TenantId, and the tenant id taken from the token.
- The API authorizes the route, and a test proves tenant A cannot read tenant B's id.

- A noisy or regulated tenant can move to a separate database. The application resolves the connection by tenant. Migrations must apply to every database, which is an operational cost I name before I offer the option.
- I do not start with a database per tenant for hundreds of small customers. The migration and the connection story will dominate the team.

Behavioral structure

Use Situation, Task, Action, Result, and a last line: what the team does now. Keep it under two minutes. I is for your decision. We is for the work the team did. Panels listen for whether you actually decided something.

> **Sample stories are scaffolds**
>
> The next three stories are examples of shape and detail. Do not memorize them as your career. Replace the system, the number, and the decision with yours. A panel that hears a generic checkout story and then asks a follow-up you cannot answer will trust you less than if you had told a smaller true story.

Sample — production incident

Situation: a release went out at 11:00 and by 11:20 the order GET p95 was above four seconds. Task: I was the tech lead on call and had to choose rollback or fix-forward. Action: I checked the deploy marker against the latency chart, saw SQL dependency time jump, and swapped back to the previous slot within fifteen minutes because the migration had only added a nullable column and the old code was safe. Then I looked at Query Store. A new screen was loading lines with a query that scanned Orders. We added a covering index in a reviewed script, load-tested the read, and shipped it that afternoon. Result: latency returned to about 200 milliseconds after the swap, and the index kept the new screen. The team rule now is that a list endpoint PR includes the filter, the index, and a sample plan, and rollback is pre-written on the release ticket.

Sample — disagreement

Situation: a senior developer wanted a generic repository and a new microservice for pricing before a release. Task: I needed a data-access approach the whole team could review, and a date we could hit. Action: I asked what change we were protecting. We had one query shape and one team. I proposed we keep pricing in a module, inject a focused IPriceService, and skip the generic repository so the SQL stayed visible. I wrote a one-page note with the consequences and I asked them to own the module boundary so the later split would be possible. Result: we shipped on time, the query stayed in one class we could tune, and the relationship held because they kept a real design ownership. I would split pricing later if a second team needed to release it.

Sample — growing someone

Situation: a developer with two years of experience was copying controller code and calling SaveChanges from actions. Task: I needed the next feature to land in a shape the team could maintain. Action: I paired on one endpoint, moved the use case into a handler, and left them to do the next one with a checklist: cancellation token, a projection for the read, a test for the failure path, and a status code. I reviewed the

rule, not the formatting. Result: they owned the returns API two months later and caught an over-posting bug in someone else's PR. The checklist is now in the repo.

Q1. Tell me about a time you missed a date.

Answer. Pick a real date. Say the original commitment, the new risk you learned, and the day you told stakeholders, which should be early. Say what you cut: a report, an edge case, a non-critical admin screen. Say what you refused to cut: auth, a migration rollback, or the payment test. Say the new date you hit and the planning change, such as estimating the first slice only. Avoid a story where you heroically worked a weekend and changed nothing about planning.

> **Say this as a tech lead**
>
> The score is whether you re-planned in public. Quietly absorbing the slip is what a strong individual contributor does. A lead changes the scope with the stakeholders while the date still means something.

Q2. How do you handle technical debt?

Answer. I name the debt in user or operator language: this list endpoint will time out at the next tenant's volume, or this Framework host is out of support in November. I attach a cost to leave it and a cost to fix it. I put a slice of the fix in the same increment as a nearby feature when the code is already open. I keep a small reserve, and I spend it on the item with the nearest incident, not on the rewrite I find interesting. I do not hide a migration inside an unrelated story. I also delete debt. A pattern we no longer need is a ticket with an owner.

> **Say this as a tech lead**
>
> Bring one debt item from your current system, with the incident it implies and the smallest fix. Vague passion about clean code is weaker than one dated risk.

Q3. How do you review code as a lead without becoming the bottleneck?

Answer. I publish review rules so people can apply them without me: cancellation, authorization, SQL shape, secrets, tests for the money path, and migration safety. I review the contract and the risk, and I let the formatter own the braces. I am available at agreed times so reviews do not sit overnight. I ask the author to walk me through a design before a thousand-line PR exists. Two approvals are required for auth and for schema. A junior's first reviews are paired so the comments teach. If every PR waits for me, I have failed, and I pick a second reviewer and a written rule to remove myself from the middle.

> **Say this as a tech lead**
>
> Say the rule you would write down this month. A panel believes a lead who has a rule. It is less convinced by a lead who says I review everything carefully.

Q4. What do you do when you disagree with your manager's technical direction?

Answer. I restate their goal until they agree I understood it. I put my alternative in writing with the risk, the date, and the operational cost. I distinguish a reversible choice from a one-way door. On a reversible choice I disagree, commit, and watch the metric we named. On a one-way door, such as a shared database across a boundary we cannot undo, or a production login as db_owner, I say that clearly and offer a smaller

experiment. I do not relitigate in the team channel after the decision. If the decision is unsafe for customers or staff, I escalate with the risk written down, not with a complaint.

Say this as a tech lead

The one-way door language is the technique. Use it once in the interview, on a real example, and then stop. Repeating the phrase sounds rehearsed.

Questions they ask you, which you should prepare

- What is the first slice you would want me to lead in the first 90 days?
- Which systems are still on .NET Framework, and is there a funded plan before support ends?
- How does a change reach production? Who can roll it back at 21:00?
- What does on-call look like, and what woke the team up last month?
- Where does the team want a tech lead to spend time: design, review, people, or still a large share of coding?
- How are SQL changes reviewed, and what rights does the application identity have today?

A lead who asks about rollback, identity, and on-call sounds like they intend to live with the system. Ask two questions, not six, at the end of a round.

Evening drill

1. Whiteboard Design A in 20 minutes. Photograph it. Look for a missing failure mode.
2. Record your incident story. Delete any sentence that does not contain a decision or a number.
3. Write the 90-day focus you want, in five lines, so you can answer if they ask.

Day 10 — Mock day and compression

Outcome: you can give a 90-second answer, survive rapid fire, and walk into the interview with a one-page sheet you wrote yourself. Today is speaking, not new theory.

Time box (4 hours)

1. Round 1, 45 min. A friend or a recorder asks any eight questions from this document. You answer in 90 seconds. They may ask one follow-up. Score each answer 0 to 2 on correct, tradeoff, and consequence.
2. Round 2, 45 min. Rapid-fire table. If you hesitate, the short answer is not yet yours. Mark it.
3. Round 3, 40 min. Design A from a blank page, 20 minutes, then a two-minute summary as if the interviewer said we are out of time.
4. Round 4, 30 min. Your incident story and your migration plan, back to back.
5. 40 min. Build the one-page sheet at the end. Anything you still miss goes on a list for the next morning, not into a new chapter tonight.
6. 20 min. Prepare two questions for them, and your positioning line.

90-second rule

- Sentence one is the decision.
- Sentence two is how it works in the API, SQL, or Azure.
- Sentence three is the tradeoff or the production consequence.
- Stop. Silence is fine. Let them pull the depth.
- If you hear yourself listing pattern names, return to the problem you were solving.

Rapid fire

Question	Short answer
401 vs 403	401 means we do not know you. 403 means we know you and you may not do this.
PUT vs PATCH vs POST	PUT replaces. PATCH changes a part. POST creates or starts work that is not simply a replace.
Transient, scoped, singleton	Per resolve, per request, per process. DbContext is scoped. A singleton must not capture a scoped service.
Why not .Result	It blocks a thread-pool thread and can starve the API under load. Await instead, and pass a CancellationToken.
IEnumerable vs IQueryable	IEnumerable runs in memory. IQueryable can become SQL. Materialize at the boundary.
AsNoTracking	Use it on reads so EF does not snapshot entities you will not update.
N+1	One query per child row, usually a loop or lazy load. Replace it with one projection or a controlled include.
rowversion	A concurrency token. The update matches the old value. Zero rows means 409, not a silent overwrite.
Clustered vs nonclustered	Clustered orders the table. Nonclustered is a secondary key path. Include columns to avoid lookups.
SARGable	The column stands alone in the predicate so an index can seek. Do not wrap the column in a function.
Deadlock 1205	A lock cycle. SQL kills a victim. Retry if safe, then remove the cycle with order, indexes, and shorter transactions.
RCSI	Readers use row versions and do not block writers the way shared locks do. It does not replace a concurrency token.
db_owner for the app	No. Custom runtime role for DML. Separate identity for migrations.
Idempotency key	A client key stored uniquely with the first response, so a retried POST does not create a second order.
Outbox	Business row and message row commit together. A relay publishes. Consumers dedupe.
Peek lock	Service Bus hides the message until you complete it. Abandon or lock expiry makes it visible again. Too many failures dead-letter it.
Cold start	A scaled-to-zero function pays startup latency on the next call. Buy a warm plan if a user is waiting.
Deployment slot	Stage and warm the new bits, swap, keep the old bits for swap-back. Schema must still suit both.
Managed identity	The Azure resource authenticates without a password. Grant it Key Vault and a SQL user in a least-privilege role.
Strangler fig	Route one slice at a time to the new host. Delete the old slice only when traffic, logs, and rollback are proven.
Generic repository	DbContext is already a unit of work. Prefer a focused store per aggregate so SQL stays visible.

Mediator	A dispatcher to one handler, useful when pipeline behaviors earn it. Handlers do not call handlers.
Circuit breaker	Fail fast when a dependency is sick, then probe. Retries need a budget or they amplify the outage.
Sync vs async integration	Sync when the user needs the answer now and you accept the dependency. Async when the work can finish after the response.
Database per service	The service owns its writes. Cross-service joins become APIs, copied data, or a reporting store.
Startup Migrate()	Unsafe with many instances. Apply a reviewed SQL script from the pipeline before traffic moves.
ConfigureAwait(false)	Important in libraries that must not resume on a captured context. ASP.NET Core app code has no sync context. Blocking calls still hurt.
record vs class	Record for value-shaped contracts. Class for entities with identity that EF tracks.
ProblemDetails	A consistent error body with status, title, and trace id. Clients and logs share it.
Keyset vs offset	Keyset seeks the next row from the last key. Offset discards skipped rows and gets worse on deep pages.
.NET support	.NET 10 is the LTS to target, supported into November 2028. .NET 8 and .NET 9 end support in November 2026.

Mock prompts — pull eight at random

1. Tell me about yourself as a backend lead.
2. Explain async to a mid-level developer who used .Result.
3. Design POST /orders so partners can retry.
4. Your list endpoint is slow. Walk me through the first hour.
5. Would you use a generic repository?
6. Two updates hit one row. What happens?
7. What SQL role does the application have, and why?
8. App Service or Functions for this API and for the email?
9. What is a dead-letter queue for?
10. Should this system be microservices?
11. How would you migrate a Framework Web API to .NET 10?
12. Tell me about a production incident.
13. Tell me about a disagreement on design.
14. What do you look for in a pull request?
15. What do you want to know about our on-call and our release path?

Scoring sheet

Question you drew	Correct 0-2	Tradeoff 0-2	Consequence 0-2	Fix tonight

Any row under 4 needs one more spoken pass tomorrow morning. Do not add a new topic because you are nervous. Depth on orders, SQL, and one migration story will beat a thin pass over every pattern name.

One-page sheet to rewrite in your handwriting

- Positioning line.
- Pipeline order: exception, HTTPS, routing, CORS, authentication, authorization, endpoints.
- Lifetimes and the captive dependency.
- Status codes you always confuse. Write only those.
- Transaction boundary: save state and outbox together; call the bus and email outside the transaction.
- Index shape for your hottest list.
- Runtime SQL role versus migration identity.
- Peek lock, complete, abandon, dead-letter, idempotency unique key.
- Strangler metric: percent of traffic, plus expand/contract rollback.
- Incident story in six lines: symptom, decision minute, action, number, rule the team has now.
- Two questions for the interviewer.

If the interview is more than ten days away

Repeat Day 10 every third day. On the days between, code one small Web API on .NET 10 if your hands have gone stale: one scoped DbContext, one migration, one concurrency token, one no-tracking projection, one function with a queue trigger, and a README that states the SQL role. Keep it small enough to finish. A lead who cannot open a solution and show the boundary will be asked to, in some panels.

The night before

- Read only your handwritten page and your two stories.
- Sleep. A rested answer with one tradeoff beats a tired tour of every Azure product.
- Prepare a short true account of your current system: one API, one database, one deploy path, one scar.
- If you do not know, say how you would find out. Then stop talking.

Closing line you can use

I still want to design the backend and read the SQL. The difference in this role is that I also own the boundary, the release, the permissions, and the way the team reviews the next change. I will leave you with a decision, the cost of that decision, and how we roll it back.

After the interview

Write down every question you were asked within an hour, while the wording is fresh. Mark the ones where you listed tools and forgot the consequence. Those are the only topics to repair before the next round. Send a short thank-you that mentions one specific technical thread, such as their migration or their on-call model, and the judgment you would bring to it. Do not attach this whole roadmap. Two paragraphs is enough.

A mentor's last note

You already have eight years. The panel is not deciding whether you can define an interface. They are deciding whether they can hand you a production API, a SQL box, a release, and a less experienced developer, and trust the outcome. Every answer in this guide is aimed at that decision. Speak in decisions, costs, and rollbacks. Use your own incidents. That is the job.